LESSON 1

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Hello, ESP32!

Meet your microcontroller, make it talk, and blink your first light.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- What a **microcontroller** is
- How to upload a **sketch** from the Arduino IDE
- How to use the **Serial Monitor** to see messages
- How to build a circuit on a **breadboard**

SESSION PLAN

0:00	Kit tour: what's in the box?
0:15	Set up the Arduino IDE
0:35	Sketch 1: **L01_Hello** (talk to the computer)
0:55	Short break
1:00	Build the LED circuit
1:20	Sketch 2: **L01_Blink**
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ 1 red LED
- ☐ 1 resistor, 220 Ω (red-red-brown)
- ☐ 2 jumper wires

1. Meet the ESP32

The ESP32 is a **microcontroller**: a tiny computer on one chip. It has no screen or keyboard. Instead it has **pins**, the metal legs along both sides of the board. Your program can switch each pin **on** (3.3 volts) or **off** (0 volts), or **read** what is connected to it.

Microcontrollers are hidden inside microwaves, toys, washing machines, cars and game controllers. The ESP32 is a powerful one: it even has **Wi-Fi** and **Bluetooth** built in.

> **Big idea: a program for a microcontroller is called a *sketch***
>
> You write the sketch on the computer in the **Arduino IDE**. Then you **upload** it down the USB cable. The ESP32 remembers it, even after you unplug it, and runs it again every time it gets power.

Find these on your ESP32 board:

- ☐ The **USB-C socket**: power and data come in here
- ☐ The **EN** (or RST) button: restarts your program
- ☐ The **BOOT** button: you only need it if uploading gets stuck
- ☐ The pin labels. Look for **GND**, **3V3** and **26** (it may say **D26** or **IO26**)

2. Set up the Arduino IDE

Your teacher may have done some of these steps already. Tick each one when it's done.

- ☐ Open the **Arduino IDE** app (version 2).
- ☐ **File** → **Preferences**. In *Additional boards manager URLs* paste:
`https://espressif.github.io/arduino-esp32/package_esp32_index.json`
- ☐ **Tools** → **Board** → **Boards Manager**. Search **esp32** and install **esp32 by Espressif Systems**.
- ☐ Plug in your ESP32 with the USB-C cable.
- ☐ **Tools** → **Board** → **esp32** → **ESP32 Dev Module**
- ☐ **Tools** → **Port**: pick the one that appeared when you plugged in (e.g. `COM3` or `/dev/cu.usbserial-…`)

> **No port showing up?**
>
> Try a different USB cable. Some cables only carry power, not data. On Windows or Mac you may need the **CP210x driver** from silabs.com. Ask your teacher.

3. Sketch 1: Make the ESP32 talk

File → **New Sketch**. Delete everything in the new window and type in the code below exactly. Capital letters matter! `Serial` is not the same as `serial`.

Then **File** → **Save As…** and name it **L01_Hello**.

Type this in → save it as **L01_Hello** — 18 lines

```
// Lesson 1: Hello, ESP32!
// The ESP32 talks to the computer over the USB cable.

int count = 0;

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("Hello! I am an ESP32.");
  Serial.println("My name is: Sparky");
}

void loop() {
  count = count + 1;
  Serial.print("I have counted to ");
  Serial.println(count);
  delay(1000);
}
```

Click the **Upload** button (the round arrow →). Wait until it says *Done uploading*. Then open **Tools** → **Serial Monitor** and set the speed in the bottom corner to **115200 baud**.

Upload stuck on *Connecting……*?

Hold down the **BOOT** button on the ESP32 until the upload starts, then let go.

How it works

Code	What it means
`void setup() { … }`	Runs **once** when the ESP32 starts
`void loop() { … }`	Runs **over and over forever**
`Serial.begin(115200);`	Start talking to the computer at 115200 "bits per second"
`Serial.println("…");`	Print a message, then start a new line
`int count = 0;`	Make a **variable**, a labelled box that holds a whole number
`count = count + 1;`	Take what's in the box, add 1, put it back
`delay(1000);`	Wait 1000 milliseconds (= 1 second)

Try it

1. Change `Sparky` to a name you choose for your ESP32. Upload again.
2. Change `delay(1000)` to `delay(100)`. What happens?
3. Press the **EN** button on the board. What happens to the count?

Red error message?

Read the line number in the error. The most common mistakes are a missing `;` at the end of a line, a missing `}` bracket, or a typo in capital letters.

4. Build your first circuit

A **breadboard** lets you connect parts without soldering. Inside, each short row of 5 holes is joined by a metal strip. The long lines along the edges (marked **+** and **–**) are the **power rails**.

Getting your breadboards ready

Clip your two breadboards together side by side. Push the ESP32 in so that its left row of pins sits in one breadboard and its right row in the other. Press gently and evenly. Now every pin has free holes next to it.

An **LED** (light emitting diode) only lets electricity flow one way. The **long leg** is **+** and goes towards the pin. The **short leg** is **–** and goes towards GND. The **resistor** limits the current so the LED doesn't burn out.

✓	From	To
☐	ESP32 pin **GND**	breadboard **–** rail (jumper wire)
☐	ESP32 pin **26**	one leg of the **220 Ω resistor**
☐	other leg of the resistor	**long leg** of the red LED
☐	**short leg** of the LED	**–** rail

Electricity flows in a loop: out of pin 26 → through the resistor → through the LED → back into GND. No loop means no light!

5. Sketch 2: Blink!

File → **New Sketch**, type this in, and save it as **L01_Blink**. Upload it.

Type this in → save it as **L01_Blink** — 15 lines

```
// Lesson 1: Blink an LED
// LED + 220 ohm resistor on pin 26.

const int LED_PIN = 26;

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);  // LED on
  delay(500);
  digitalWrite(LED_PIN, LOW);   // LED off
  delay(500);
}
```

Code	What it means
`const int LED_PIN = 26;`	A **constant**: a name for the number 26 that never changes
`pinMode(LED_PIN, OUTPUT);`	Pin 26 will *send out* electricity
`digitalWrite(LED_PIN, HIGH);`	Switch the pin **on** (3.3 V)
`digitalWrite(LED_PIN, LOW);`	Switch the pin **off** (0 V)
`// LED on`	A **comment**. The ESP32 ignores everything after `//`

LED not lighting?

Turn the LED around: the long leg must go to the resistor. Check the GND wire is in the **–** rail. Check you used pin **26**.

6. Challenges

Challenge 1: Speed it up

Make the LED blink really fast, then really slowly. What is the fastest blink your eyes can still see?

Challenge 2: Heartbeat

Make the LED go *blink-blink……pause, blink-blink……pause* like a heartbeat.

Challenge 3: SOS

In Morse code, SOS is · · · – – – · · · (3 short, 3 long, 3 short). Make your LED flash SOS. *Hint: a short flash could be 200 ms and a long flash 600 ms.*

Bonus: Blink and talk

Combine both sketches. Print `"ON"` and `"OFF"` in the Serial Monitor every time the LED changes.

7. Quiz

Check what you learned

1. What is the difference between `setup()` and `loop()`?

2. Which leg of an LED is the **+** leg?

3. How many milliseconds are in 3 seconds?

Tidy up: unplug the USB cable. You can leave your LED circuit on the breadboard for next time.

LESSON 2

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Traffic Lights

Control three LEDs, make them take turns, and write your own function.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How to control **several pins** at once
- How to plan a **sequence** of steps
- How to write your own **function**
- How an **array** and a **for loop** save typing

SESSION PLAN

0:00	Warm-up: how does a real traffic light work?
0:10	Build the three-LED circuit
0:30	Sketch 1: **L02_TrafficLight**
0:55	Short break
1:00	Sketch 2: **L02_LightFunction** (your own function)
1:20	Sketch 3: **L02_Chase** (arrays and for loops)
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ 1 red, 1 yellow and 1 green LED
- ☐ 3 resistors, 220 Ω (red-red-brown)
- ☐ 4 jumper wires

1. Warm-up: think like a traffic engineer

A traffic light is a little computer that follows a **sequence**: the same steps, in the same order, forever. Before you write any code, write down the steps. Number them, and say which lights are on and for how long.

Plan your traffic light

Write each step: which colours are on, and how many seconds it lasts.

_____

_____

_____

In this lesson we will use **red** → **green** → **yellow** → **back to red**. Some countries do it differently. You'll get to change it in a challenge.

2. Build the circuit

This is the same idea as the LED from Lesson 1, done three times. Each LED gets its own pin and its own resistor. All three short legs share the **–** rail.

✓	From	To
☐	ESP32 pin **GND**	breadboard **–** rail
☐	ESP32 pin **25**	220 Ω resistor → **long leg** of the **red** LED
☐	ESP32 pin **26**	220 Ω resistor → **long leg** of the **yellow** LED
☐	ESP32 pin **27**	220 Ω resistor → **long leg** of the **green** LED
☐	**short legs** of all 3 LEDs	**–** rail

Keep it tidy

Put the LEDs in a line, red at the top, like a real traffic light. Leave a gap between them so the legs don't touch.

3. Sketch 1: A traffic light

File → **New Sketch**, type this in, and save it as **L02_TrafficLight**. Upload it.

Type this in → save it as **L02_TrafficLight** — 32 lines

```
// Lesson 2: Traffic light
// Red LED on 25, yellow on 26, green on 27.

const int RED = 25;
const int YELLOW = 26;
const int GREEN = 27;

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(YELLOW, OUTPUT);
  pinMode(GREEN, OUTPUT);
}

void loop() {
  // Stop
  digitalWrite(RED, HIGH);
  digitalWrite(YELLOW, LOW);
  digitalWrite(GREEN, LOW);
  delay(3000);

  // Go
  digitalWrite(RED, LOW);
  digitalWrite(YELLOW, LOW);
  digitalWrite(GREEN, HIGH);
```

```
  delay(3000);

  // Get ready to stop
  digitalWrite(RED, LOW);
  digitalWrite(YELLOW, HIGH);
  digitalWrite(GREEN, LOW);
  delay(1000);
}
```

How it works

Code	What it means
`const int RED = 25;`	Give pin 25 a name we can remember
`pinMode(RED, OUTPUT);`	Each LED pin needs its own `pinMode`
three `digitalWrite` lines	Set **every** light for this step, even the ones that stay off
`delay(3000);`	Hold this step for 3 seconds

> **Big idea: the computer does exactly what you say**
>
> Why do we switch off lights that are already off? Because each step should say *exactly* what the light looks like. Then you can change the order of the steps and nothing goes wrong.

> **Try it**
>
> 1. Make the green light last twice as long.
> 2. Add a fourth step: all three lights off for half a second.

4. Sketch 2: Write your own function

Sketch 1 has lots of lines that look almost the same. Programmers hate typing the same thing again and again! A **function** is a set of instructions with a name. We'll make one called `lights` that sets all three LEDs and then waits.

File → Save As… and name it **L02_LightFunction**. Then change your code to this:

Type this in → save it as **L02_LightFunction** — 25 lines

```
// Lesson 2: Traffic light with our own function
// Same wiring as L02_TrafficLight.

const int RED = 25;
const int YELLOW = 26;
const int GREEN = 27;

void lights(int r, int y, int g, int wait) {
  digitalWrite(RED, r);
  digitalWrite(YELLOW, y);
  digitalWrite(GREEN, g);
```

```
  delay(wait);
}

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(YELLOW, OUTPUT);
  pinMode(GREEN, OUTPUT);
}

void loop() {
  lights(HIGH, LOW, LOW, 3000);   // stop
  lights(LOW, LOW, HIGH, 3000);   // go
  lights(LOW, HIGH, LOW, 1000);   // get ready to stop
}
```

Code	What it means
`void lights(int r, int y, int g, int wait)`	Make a function called `lights`. It needs 4 numbers, called **parameters**
`digitalWrite(RED, r);`	Use whatever was passed in as `r` (`HIGH` or `LOW`)
`lights(HIGH, LOW, LOW, 3000);`	**Call** the function: red on, yellow off, green off, wait 3 s
`void`	This function does something but doesn't give back an answer

The light works exactly the same as before, but `loop()` is now 3 lines long and easy to read.

> **Same order every time**
>
> The values go into the parameters **in order**. In `lights(LOW, HIGH, LOW, 1000)` the first value goes to `r`, the second to `y`, the third to `g`, and the last to `wait`.

5. Sketch 3: Light chase

Now we'll stop being a traffic light and make a running light, like on a game show. **File → New Sketch**, save it as **L02_Chase**.

Type this in → save it as **L02_Chase** — 19 lines

```
// Lesson 2: Light chase
// Same wiring. A light runs along the LEDs.

int leds[] = {25, 26, 27};
int speed = 150;

void setup() {
  for (int i = 0; i < 3; i++) {
    pinMode(leds[i], OUTPUT);
  }
}
```

```
void loop() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(leds[i], HIGH);
    delay(speed);
    digitalWrite(leds[i], LOW);
  }
}
```

Code	What it means
`int leds[] = {25, 26, 27};`	An **array**: one name holding a list of numbers
`leds[0]`	The **first** item in the list (25). Counting starts at **0**!
`for (int i = 0; i < 3; i++)`	Repeat the `{ }` block with `i` = 0, then 1, then 2
`i++`	Add 1 to `i`

Big idea: loops do the boring work

The `for` loop in `setup()` runs `pinMode` three times for you. With 30 LEDs you would only change the `3` and the array, not write 30 lines.

Try it

1. Change `speed` to `50`. Then `500`.
2. Delete the line `digitalWrite(leds[i], LOW);`. What happens? Why?

6. Challenges

Challenge 1: UK traffic light

In the UK, red and yellow come on **together** before green. Change **L02_LightFunction** to do red → red + yellow → green → yellow.

Challenge 2: Night mode

At night some traffic lights just flash yellow. Make a sketch that flashes the yellow light on and off, with the others off.

Challenge 3: Bouncing light

Change **L02_Chase** so the light goes 25 → 26 → 27 → 26 → 25 → … and bounces back and forth.
Hint: add a second `for` loop that counts down with `i--`.

Bonus: All together now

Make all three LEDs blink together 3 times, then do one chase, then repeat. Write a function for each pattern.

7. Quiz

Check what you learned

1. What is a **function**, and why is it useful?

2. In `int leds[] = {25, 26, 27};` what is `leds[2]`?

3. How many times does `for (int i = 0; i < 5; i++)` repeat?

Tidy up: unplug the USB cable. Keep your three LEDs on the breadboard: you'll use them again in Lesson 3.

LESSON 3

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Push the Button

Give your ESP32 an input, so it can react to you.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- The difference between **input** and **output**
- How to read a **button** with `digitalRead`
- How to make decisions with **if / else**
- How to remember things with a **bool** variable
- How a **tilt switch** can sense shaking

SESSION PLAN

0:00	Warm-up: inputs and outputs all around us
0:10	Add a button to your circuit
0:20	Sketch 1: **L03_Button**
0:35	Sketch 2: **L03_Toggle** (on/off switch)
0:55	Short break
1:00	Sketch 3: **L03_Counter** (count presses)
1:20	Sketch 4: **L03_ShakeAlarm** (tilt switch)
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Your traffic light LEDs from Lesson 2 (red 25, yellow 26, green 27)
- ☐ 2 push buttons
- ☐ 1 tilt ball switch
- ☐ 4 jumper wires

1. Inputs and outputs

So far the ESP32 has only **talked**: it switched LEDs on and off. Those are **outputs**. Today it will **listen**. A button is an **input**: the ESP32 checks whether it is pressed, then decides what to do.

> **Spot the inputs**
>
> Think of a TV remote, a game controller or a microwave. Name two **inputs** and two **outputs**.
>
> _____
>
> _____

2. Add a button

A push button has 4 legs. Inside, the legs are joined in **pairs**. Pressing the button connects the two pairs together.

> **The diagonal trick**
>
> Put the button across the middle gap of a breadboard. Then use two legs that are **diagonally opposite**. That way you always get one leg from each pair, whichever way round the button is.

Keep your traffic light LEDs from Lesson 2 in place, then add:

✓	From	To
☐	Push button	across the middle gap of a breadboard
☐	ESP32 pin **13**	one leg of the button
☐	the **diagonally opposite** leg	**–** rail (GND)

That's it: no resistor needed! Read on to find out why.

3. Sketch 1: Light while pressed

File → **New Sketch**, type this in, save it as **L03_Button**, and upload it. Hold the button down: the yellow LED should light up.

Type this in → save it as **L03_Button** — 18 lines

```
// Lesson 3: Press a button to light an LED
// Button on pin 13 (other leg to GND), yellow LED on 26.

const int BUTTON = 13;
const int LED = 26;

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  if (digitalRead(BUTTON) == LOW) {
    digitalWrite(LED, HIGH);   // pressed
  } else {
    digitalWrite(LED, LOW);    // not pressed
  }
}
```

How it works

Code	What it means
`pinMode(BUTTON, INPUT_PULLUP);`	Pin 13 is an **input**, and the ESP32 gently pulls it up to **HIGH**
`digitalRead(BUTTON)`	Read the pin: `HIGH` or `LOW`
`== LOW`	**Is it equal to** LOW? Note the **two** equals signs!
`if ( … ) { … } else { … }`	If the test is true, do the first block. Otherwise, do the `else` block

Big idea: pull-up means pressed = LOW

With `INPUT_PULLUP`, the pin reads **HIGH** when nothing is happening. Pressing the button connects the pin to GND, so it reads **LOW**. It feels backwards, but it means we don't need an extra resistor.

`=` is not `==`

One `=` **puts** a value in a variable: `count = 5;`. Two `==` **compare** two values: `if (count == 5)`. Mixing them up is the most common bug in all of programming!

4. Sketch 2: An on/off switch

Holding the button down is tiring. Let's make it work like a light switch: press once for **on**, press again for **off**. To do that, the ESP32 has to **remember** whether the light is on.

File → New Sketch, save it as **L03_Toggle**.

Type this in → save it as **L03_Toggle** — 25 lines

```
// Lesson 3: Press once for on, press again for off
// Same wiring as L03_Button.

const int BUTTON = 13;
const int LED = 26;

bool ledOn = false;
bool wasPressed = false;

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  bool pressed = digitalRead(BUTTON) == LOW;

  if (pressed && !wasPressed) {
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
  }

  wasPressed = pressed;
  delay(20);   // ignore the bouncing contacts
}
```

Code	What it means
`bool ledOn = false;`	A **bool** holds only `true` or `false`. It is our memory of the LED
`pressed && !wasPressed`	`&&` means **and**. `!` means **not**. So: pressed now, and *not* pressed last time
`ledOn = !ledOn;`	Flip it: `true` becomes `false`, `false` becomes `true`
`wasPressed = pressed;`	Remember the button for next time round the loop

Big idea: the moment of pressing

`loop()` runs thousands of times a second. If we flipped the LED every time the button *is* down, it would flicker crazily. We only flip at the moment the button **changes** from up to down.

Try it

Delete the `delay(20);` line and upload. Press the button many times. Does it sometimes miss or flip twice? Buttons **bounce**: the metal contacts rattle for a few milliseconds. The short delay makes the ESP32 ignore the rattle. Put the line back!

5. Sketch 3: Count the presses

Add a second button, the same way as the first:

✓	From	To
☐	ESP32 pin **14**	one leg of the second button
☐	the **diagonally opposite** leg	**–** rail (GND)

File → **New Sketch**, save it as **L03_Counter**. Upload, then open the **Serial Monitor** (115200 baud).

Button 13 adds one; button 14 resets the count to zero. The `delay(300)` after a reset stops it printing *Reset to 0!* hundreds of times while you hold the button.

Type this in → save it as **L03_Counter** — 33 lines

```
// Lesson 3: Count button presses
// Count button on 13, reset button on 14 (both to GND).

const int COUNT_BTN = 13;
const int RESET_BTN = 14;

int count = 0;
bool wasPressed = false;

void setup() {
  Serial.begin(115200);
  pinMode(COUNT_BTN, INPUT_PULLUP);
  pinMode(RESET_BTN, INPUT_PULLUP);
}

void loop() {
```

```
  bool pressed = digitalRead(COUNT_BTN) == LOW;

  if (pressed && !wasPressed) {
    count = count + 1;
    Serial.print("Presses: ");
    Serial.println(count);
  }
  wasPressed = pressed;

  if (digitalRead(RESET_BTN) == LOW) {
    count = 0;
    Serial.println("Reset to 0!");
    delay(300);
  }

  delay(20);
}
```

> **Try it**
>
> How many presses can you do in 10 seconds? Get a friend to time you!

6. Sketch 4: Shake alarm

A **tilt ball switch** has a tiny metal ball inside. When it's upright, the ball touches the contacts and the switch is **closed**. Tip it or shake it, and the ball rolls away: the switch **opens**. It works just like a button that is pressed by gravity!

Take out the second button (pin 14) and put the tilt switch in its place:

✓	From	To
☐	ESP32 pin **14**	one leg of the tilt switch (either way round)
☐	other leg of the tilt switch	**–** rail (GND)

File → New Sketch, save it as **L03_ShakeAlarm**.

Type this in → save it as **L03_ShakeAlarm** — 31 lines

```
// Lesson 3: Shake alarm
// Tilt switch on pin 14 (other leg to GND), red LED on 25.

const int TILT = 14;
const int ALARM_LED = 25;

int lastTilt;
int shakes = 0;

void setup() {
  Serial.begin(115200);
  pinMode(TILT, INPUT_PULLUP);
```

```
  pinMode(ALARM_LED, OUTPUT);
  lastTilt = digitalRead(TILT);
}

void loop() {
  int tilt = digitalRead(TILT);

  if (tilt != lastTilt) {
    shakes = shakes + 1;
    Serial.print("Shake detected! Total: ");
    Serial.println(shakes);

    digitalWrite(ALARM_LED, HIGH);
    delay(1000);
    digitalWrite(ALARM_LED, LOW);

    lastTilt = digitalRead(TILT);
  }
}
```

Code	What it means
`int lastTilt;`	Remember what the switch was last time
`tilt != lastTilt`	`!=` means **is not equal to**. Something changed: it was shaken!
`lastTilt = digitalRead(TILT);`	After the alarm, read it again so we start fresh

7. Challenges

Challenge 1: Two-button control

Use button 13 to turn the green LED **on** and button 14 to turn it **off**.

Challenge 2: Pedestrian crossing

Combine Lesson 2 and today. Cars get a green light, until somebody presses the button. Then the light goes yellow → red for 5 seconds → back to green. *Hint: copy the* `lights()` *function from L02_LightFunction.*

Challenge 3: Reaction game for two

The yellow LED switches on after a random wait (`delay(random(2000, 6000));`). Player 1 has button 13, player 2 has button 14. Whoever presses first wins, and their LED (red or green) lights up.

Bonus: Secret knock

The green LED only lights up if you press button 13 exactly **3 times** and then button 14.

8. Quiz

Check what you learned

1. With `INPUT_PULLUP`, what does `digitalRead` give when the button is pressed?

2. What is the difference between `=` and `==`?

3. What values can a `bool` variable hold?

Tidy up: unplug the USB cable. Put the tilt switch back in the kit. Keep the LEDs and the buttons on 13 and 14 for later lessons.

LESSON 4

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Knobs and Light Sensors

Measure "how much" instead of just on or off.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- The difference between **digital** and **analog**
- How to read a knob with `analogRead`
- How to draw live graphs with the **Serial Plotter**
- How to change ranges of numbers with `map()`
- How to sense light with a **photoresistor**

SESSION PLAN

0:00	Warm-up: digital vs analog
0:10	Wire up the potentiometer
0:20	Sketch 1: **L04_ReadKnob** + Serial Plotter
0:40	Sketch 2: **L04_SpeedKnob** (`map()`)
0:55	Short break
1:00	Wire up the photoresistor
1:10	Sketch 3: **L04_NightLight**
1:30	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Your LEDs from Lesson 2 (red 25, yellow 26, green 27)
- ☐ 1 potentiometer (10 kΩ knob)
- ☐ 1 photoresistor (wiggly line on top)
- ☐ 1 resistor, 10 kΩ (brown-black-orange)
- ☐ 5 jumper wires

1. Digital vs analog

A button is **digital**: it's either pressed or not. Only two answers: `HIGH` or `LOW`. But lots of things in the world aren't just on or off. How loud is it? How hot? How bright? These are **analog** values: they can be anything in between.

The ESP32 has special pins that can measure **voltage**, from 0 V up to 3.3 V. `analogRead` turns the voltage into a number from **0** to **4095**.

Voltage on the pin	`analogRead` gives
0 V	0
about 1.65 V (half)	about 2048
3.3 V	4095

Digital or analog?

Sort these: a light switch, a volume knob, a thermometer, a doorbell.

2. Wire up the potentiometer

A **potentiometer** (or "pot") is a knob. It has three legs. The two outer legs go to 3.3 V and GND. The middle leg slides between them as you turn the knob, so its voltage goes from 0 V to 3.3 V.

Keep your LEDs from Lesson 2. You can remove the buttons if you need room.

✓	From	To
☐	ESP32 pin **3V3**	breadboard **+** rail
☐	Potentiometer, in the breadboard	each leg in its own row
☐	pot **left** leg	**+** rail (3.3 V)
☐	pot **right** leg	**–** rail (GND)
☐	pot **middle** leg	ESP32 pin **34**

> **3V3, never 5V!**
>
> The ESP32's pins can only handle **3.3 volts**. Always connect the **+** rail to **3V3**, not to the pin marked **5V** or **VIN**. Too much voltage can break the ESP32.

3. Sketch 1: Read the knob

File → New Sketch, save it as **L04_ReadKnob**, and upload. Open the **Serial Monitor** and turn the knob.

Type this in → save it as **L04_ReadKnob** — 15 lines

```
// Lesson 4: Read the knob
// Potentiometer: outer legs to 3V3 and GND, middle leg to pin 34.

const int KNOB = 34;

void setup() {
  Serial.begin(115200);
}

void loop() {
  int value = analogRead(KNOB);
  Serial.print("knob:");
  Serial.println(value);
  delay(50);
}
```

Code	What it means
`analogRead(KNOB)`	Measure the voltage on pin 34. Gives a number from 0 to 4095
`int value = …;`	Make a new variable and put the answer straight in it
`Serial.print("knob:");`	A **label** for the plotter

Now close the Serial Monitor and open **Tools → Serial Plotter**. Turn the knob slowly, then quickly. You are drawing a live graph!

> **Pin 34 is input-only**
>
> Pins 34, 35, 36 and 39 can only **read**, never write. That makes them perfect for sensors. We use them for analog inputs throughout this course.

4. Sketch 2: Speed knob

What if the knob could control the blink speed of an LED? The knob gives 0–4095, but we want a delay between 50 ms (fast) and 1000 ms (slow). The `map()` function converts one range into another.

File → New Sketch, save it as **L04_SpeedKnob**.

Type this in → save it as **L04_SpeedKnob** — 22 lines

```
// Lesson 4: The knob sets the blink speed
// Knob on pin 34, yellow LED on pin 26.

const int KNOB = 34;
const int LED = 26;

void setup() {
  Serial.begin(115200);
  pinMode(LED, OUTPUT);
}

void loop() {
  int value = analogRead(KNOB);
  int wait = map(value, 0, 4095, 50, 1000);
  Serial.print("wait:");
  Serial.println(wait);

  digitalWrite(LED, HIGH);
  delay(wait);
  digitalWrite(LED, LOW);
  delay(wait);
}
```

Code	What it means
`map(value, 0, 4095, 50, 1000)`	Turn a number between 0 and 4095 into the matching one between 50 and 1000
`int wait = …;`	Store the answer for the two `delay`s

> **Big idea: map() is like a ruler conversion**
>
> Halfway along one ruler is halfway along the other. So 2048 (half of 4095) becomes about 525 (half way from 50 to 1000). `map()` does the maths for you.

> **Try it**
>
> 1. Swap 50 and 1000 in the `map()` line. What changes?
> 2. Make the fastest blink even faster.

5. Wire up the photoresistor

A **photoresistor** changes its resistance with light: lots of light = low resistance, darkness = high resistance. The ESP32 can't measure resistance directly. So we pair it with a fixed **10 kΩ resistor** to make a **voltage divider**: a voltage that changes as the light changes.

✓	From	To
☐	one leg of the photoresistor	**+** rail (3.3 V)
☐	other leg of the photoresistor	ESP32 pin **35**
☐	10 kΩ resistor (brown-black-orange)	from the pin **35** row to the **–** rail

Bright light → higher number. Dark → lower number.

6. Sketch 3: Automatic night light

File → New Sketch, save it as **L04_NightLight**. Upload and open the **Serial Plotter**.

Type this in → save it as **L04_NightLight** — 25 lines

```
// Lesson 4: Automatic night light
// Photoresistor from 3V3 to pin 35, 10k resistor from 35 to GND.
// The green LED on 27 comes on when it gets dark.

const int LIGHT_SENSOR = 35;
const int LED = 27;
const int DARK = 1500;   // change this to suit your room

void setup() {
  Serial.begin(115200);
  pinMode(LED, OUTPUT);
}

void loop() {
  int light = analogRead(LIGHT_SENSOR);
  Serial.print("light:");
  Serial.println(light);
```

```
  if (light < DARK) {
    digitalWrite(LED, HIGH);
  } else {
    digitalWrite(LED, LOW);
  }
  delay(100);
}
```

First, find your numbers. Watch the plotter, then cover the sensor with your hand. Write down what you see:

Room light: _____ **Covered with hand:** _____

Pick a number between the two, and put it in the line `const int DARK = 1500;`. Upload again. Now the green LED should come on when you cover the sensor.

> **Big idea: calibration**
>
> Every room and every sensor is a bit different. Real engineers measure first, then choose their settings. This is called **calibration**.

7. Challenges

> **Challenge 1: Dimmer warning**
>
> Make the night light also print `"It's getting dark!"` in the Serial Monitor, but only when it is dark.

> **Challenge 2: Level meter**
>
> Use the knob to light up 0, 1, 2 or 3 LEDs, like a volume meter: green, then yellow, then red. *Hint:* `analogRead(KNOB) / 1024` *gives 0, 1, 2 or 3.*

> **Challenge 3: Secret box alarm**
>
> Put your circuit in a dark box or drawer. If somebody opens it and light gets in, flash the red LED fast and print an alarm message.

> **Bonus: Light-speed blink**
>
> Use `map()` so the **light** controls the blink speed, not the knob. Brighter = faster.

8. Quiz

Check what you learned

1. What range of numbers does `analogRead` give on the ESP32?

2. What does `map(value, 0, 4095, 0, 100)` give when `value` is 4095?

3. Why do we need the 10 kΩ resistor with the photoresistor?

Tidy up: unplug the USB cable. Keep the potentiometer and LEDs on the breadboard: you'll need them in Lesson 5.

LESSON 5

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Rainbow Colors

Dim LEDs smoothly and mix any colour you like.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How **PWM** fakes "half on" with very fast blinking
- How to dim an LED with `analogWrite`
- How to fade with **for loops** that count up and down
- How an **RGB LED** mixes red, green and blue light

SESSION PLAN

0:00	Warm-up: how do screens make colours?
0:10	PWM: blinking too fast to see
0:20	Sketch 1: **L05_Fade**
0:40	Build the RGB LED circuit
0:55	Short break
1:00	Sketch 2: **L05_ColorMixer**
1:20	Sketch 3: **L05_KnobColor**
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Your yellow LED on pin 26 (from Lesson 2)
- ☐ 1 RGB LED (4 legs)
- ☐ 3 resistors, 220 Ω (red-red-brown)
- ☐ Your potentiometer on pin 34 (from Lesson 4)
- ☐ 4 jumper wires

1. Blinking too fast to see

`digitalWrite` can only switch an LED fully **on** or fully **off**. So how do we make it **half** bright?

The trick is to blink it **very fast**: thousands of times a second. Your eyes can't follow that, so they see the *average*. On for half the time looks half bright. On for a tenth of the time looks very dim. This trick is called **PWM** (Pulse Width Modulation).

On for…	Looks…	`analogWrite` value
0% of the time	off	0
25% of the time	dim	64
50% of the time	half bright	128
100% of the time	fully on	255

Big idea: 0 to 255

`analogWrite(pin, value)` takes a value from **0** (off) to **255** (fully on). Why 255? It's the biggest number that fits in 8 bits (one byte) of computer memory.

2. Sketch 1: Fade in, fade out

Keep the yellow LED on pin 26 from Lesson 2. **File → New Sketch**, save it as **L05_Fade**, and upload.

Type this in → save it as **L05_Fade** — 19 lines

```
// Lesson 5: Fade an LED in and out
// Yellow LED on pin 26 (from Lesson 2).

const int LED = 26;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  for (int brightness = 0; brightness <= 255; brightness++) {
    analogWrite(LED, brightness);
    delay(5);
  }
  for (int brightness = 255; brightness >= 0; brightness--) {
    analogWrite(LED, brightness);
    delay(5);
  }
}
```

How it works

Code	What it means
`analogWrite(LED, brightness);`	Set the LED's brightness, from 0 to 255
`for (int brightness = 0; brightness <= 255; brightness++)`	Count **up** from 0 to 255
`for (int brightness = 255; brightness >= 0; brightness--)`	Count **down** from 255 to 0
`brightness--`	Take 1 away from `brightness`
`delay(5);`	Wait a tiny bit at each step, so the fade is slow enough to see

Try it

1. How long does one fade up take? *Hint: 256 steps × 5 ms.*

---

2. Change `delay(5)` to `delay(1)`, then to `delay(20)`. Which looks best?

3. Build the RGB LED circuit

An **RGB LED** is really three tiny LEDs in one: **R**ed, **G**reen and **B**lue. It has four legs. The **longest** leg is the shared **–** leg (called the common cathode). It goes to GND.

> **Which leg is which?**
>
> Hold the LED with the longest leg second from the left. From left to right the legs are usually: **Red**, **GND (longest)**, **Green**, **Blue**. If your colours come out mixed up, just swap the wires on pins 25, 26 and 27.

Take out your three traffic light LEDs (but leave their resistors if you like) and add the RGB LED:

✓	From	To
☐	RGB LED	each leg in its own row
☐	**longest** leg	**–** rail (GND)
☐	ESP32 pin **25**	220 Ω resistor → **red** leg
☐	ESP32 pin **26**	220 Ω resistor → **green** leg
☐	ESP32 pin **27**	220 Ω resistor → **blue** leg

Keep the potentiometer on pin 34 from Lesson 4. You'll need it in Sketch 3.

4. Sketch 2: Colour mixer

Screens make every colour by mixing red, green and blue light. Now you can do that too! **File** → **New Sketch**, save it as **L05_ColorMixer**.

Type this in → save it as **L05_ColorMixer** — 33 lines

```
// Lesson 5: Mix colours with an RGB LED
// RGB LED: red 25, green 26, blue 27 (220 ohm each), long leg GND.

const int RED = 25;
const int GREEN = 26;
const int BLUE = 27;

void setColor(int r, int g, int b) {
  analogWrite(RED, r);
  analogWrite(GREEN, g);
  analogWrite(BLUE, b);
}

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(GREEN, OUTPUT);
  pinMode(BLUE, OUTPUT);
}
```

```
void loop() {
  setColor(255, 0, 0);      // red
  delay(1000);
  setColor(0, 255, 0);      // green
  delay(1000);
  setColor(0, 0, 255);      // blue
  delay(1000);
  setColor(255, 255, 0);    // yellow
  delay(1000);
  setColor(128, 0, 255);    // purple
  delay(1000);
  setColor(255, 255, 255);  // white
  delay(1000);
}
```

Code	What it means
`void setColor(int r, int g, int b)`	Our own function. It sets all three colours in one go
`setColor(255, 255, 0);`	Full red + full green + no blue = **yellow**
`setColor(128, 0, 255);`	Half red + full blue = **purple**

Hard to see the colours?

Put a small piece of white paper or a ping-pong ball over the LED. It blends the three colours together.

Colour lab

Try these mixes and write down what colour you see: `(0, 255, 255)` `(255, 0, 255)` `(255, 80, 0)` `(20, 20, 20)`

5. Sketch 3: Knob colour picker

Now let the knob choose the colour. As you turn it, the colour moves around the **colour wheel**: red → green → blue → back to red.

File → Save As… and name it **L05_KnobColor**. Change your code to the sketch below.

The knob gives a number 0–764, split into three parts of 255:

`pos`	Colour change
0 – 254	red fades out while green fades in
255 – 509	green fades out while blue fades in
510 – 764	blue fades out while red fades in

`else if` lets you test several things in a row: the first test that is true wins, and the rest are skipped.

Type this in → save it as **L05_KnobColor** 34 lines

```
// Lesson 5: Turn the knob to pick a colour
// RGB LED on 25/26/27, potentiometer middle leg on 34.

const int RED = 25;
const int GREEN = 26;
const int BLUE = 27;
const int KNOB = 34;

void setColor(int r, int g, int b) {
  analogWrite(RED, r);
  analogWrite(GREEN, g);
  analogWrite(BLUE, b);
}

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(GREEN, OUTPUT);
  pinMode(BLUE, OUTPUT);
}

void loop() {
  int pos = map(analogRead(KNOB), 0, 4095, 0, 764);

  if (pos < 255) {
    setColor(255 - pos, pos, 0);          // red to green
  } else if (pos < 510) {
    pos = pos - 255;
    setColor(0, 255 - pos, pos);          // green to blue
  } else {
    pos = pos - 510;
    setColor(pos, 0, 255 - pos);          // blue to red
  }
  delay(20);
}
```

6. Challenges

Challenge 1: Traffic light, RGB style

Use `setColor` to make the RGB LED act like a traffic light: red, green, yellow.

Challenge 2: Automatic rainbow

Without the knob, make the colour go round the colour wheel by itself, smoothly. *Hint: use three* `for` *loops, one for each part of the wheel.*

Challenge 3: Breathing light

Make the RGB LED "breathe": slowly fade in and out in one colour. Then make a button on pin 13 change which colour it breathes in.

Bonus: Mood lamp

Use the photoresistor from Lesson 4: when the room gets dark, the RGB LED fades on in a warm orange colour.

7. Quiz

Check what you learned

1. What does `analogWrite(LED, 0)` do? And `analogWrite(LED, 255)`?

2. In your own words, how does PWM make an LED look dim?

3. Which `setColor` values give pure blue?

Tidy up: unplug the USB cable. Keep the RGB LED and the potentiometer in place if you have room. Put the three traffic light LEDs back in the kit.

LESSON 6

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Make Some Noise

Beeps, sirens, songs, and a piano you build yourself.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- The difference between an **active** and a **passive** buzzer
- How sound is made by **vibrations** (frequency)
- How to play notes with `tone()` and `noTone()`
- How to store a song in two **arrays**

SESSION PLAN

0:00	Warm-up: what is sound?
0:10	Sketch 1: **L06_Beep** (active buzzer)
0:25	Swap to the passive buzzer, learn `tone()`
0:35	Sketch 2: **L06_Melody**
0:55	Short break
1:00	Build the 5-button keyboard
1:15	Sketch 3: **L06_Piano**
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ 1 active buzzer (sealed bottom, sticker on top)
- ☐ 1 passive buzzer (open bottom, you can see the green board)
- ☐ 5 push buttons
- ☐ About 8 jumper wires

1. What is sound?

Sound is air **vibrating**. A speaker or buzzer pushes the air back and forth very fast, and your ear feels those pushes. The number of pushes per second is the **frequency**, measured in **hertz** (Hz).

Frequency	Sounds like
100 Hz	a low hum
440 Hz	the note A, used to tune instruments
2000 Hz	a high beep
15000 Hz	a whine that only young ears can hear!

Your kit has two kinds of buzzer:

	Active buzzer	Passive buzzer
Looks like	Sealed bottom, often a sticker on top	Open bottom, you can see a green board
Inside	Its own little beep-maker	Just the part that vibrates
How to use it	Switch it on: it beeps (one sound only)	You make the vibrations: any note you like

2. Sketch 1: Beep!

Find the **active** buzzer. Its **+** leg is the longer one (there's often a **+** on top too).

✓	From	To
☐	Active buzzer, in the breadboard	each leg in its own row
☐	ESP32 pin **33**	buzzer **+** leg (longer)
☐	buzzer other leg	**–** rail (GND)

File → **New Sketch**, save it as **L06_Beep**, and upload.

Type this in → save it as **L06_Beep** — 22 lines

```
// Lesson 6: Beep with the active buzzer
// Active buzzer: + leg to pin 33, other leg to GND.

const int BUZZER = 33;

void beep(int length) {
  digitalWrite(BUZZER, HIGH);
  delay(length);
  digitalWrite(BUZZER, LOW);
  delay(150);
}

void setup() {
  pinMode(BUZZER, OUTPUT);
}

void loop() {
  beep(100);
  beep(100);
  beep(400);
  delay(2000);
}
```

This is just like blinking an LED, but with sound. The `beep()` function takes one number, the **length** of the beep in milliseconds.

> **Try it**
>
> Change the `beep` lines in `loop()` to beep your name in Morse code, or make a doorbell sound.

3. The passive buzzer and tone()

Now swap the active buzzer for the **passive** one, in exactly the same place (**+** leg to pin 33).

> **Experiment**
>
> Upload **L06_Beep** again with the passive buzzer. What do you hear? Why?

The passive buzzer only **clicks** when the pin switches on or off. To make a note, the pin has to switch on and off hundreds of times a second. The `tone()` command does that for you:

Code	What it means
`tone(BUZZER, 440);`	Vibrate pin 33 at 440 Hz (the note A), until we say stop
`noTone(BUZZER);`	Stop the sound

Here are the notes you'll need today:

Note	C	D	E	F	G	A	B	high C
Frequency (Hz)	262	294	330	349	392	440	494	523

4. Sketch 2: Play a melody

A song is a list of **notes** and how long each note lasts (its **beats**). We'll keep each list in its own array. **File → New Sketch**, save it as **L06_Melody**.

Type this in → save it as **L06_Melody** — 23 lines

```
// Lesson 6: Play a tune on the passive buzzer
// Passive buzzer: + leg to pin 33, other leg to GND.

const int BUZZER = 33;
const int TEMPO = 300;

int notes[] = {262, 262, 392, 392, 440, 440, 392,
               349, 349, 330, 330, 294, 294, 262};
int beats[] = {1, 1, 1, 1, 1, 1, 2,
               1, 1, 1, 1, 1, 1, 2};

void setup() {
}

void loop() {
  for (int i = 0; i < 14; i++) {
    tone(BUZZER, notes[i]);
    delay(beats[i] * TEMPO);
    noTone(BUZZER);
    delay(50);
  }
  delay(2000);
}
```

Code	What it means
`int notes[] = { … };`	The frequency of each note, in order
`int beats[] = { … };`	How many beats each note lasts (1 = short, 2 = long)
`const int TEMPO = 300;`	One beat lasts 300 ms. Smaller = faster song
`notes[i]` and `beats[i]`	The same `i` gives a note **and** its length
`delay(50);`	A tiny gap so repeated notes don't blur together

Big idea: data and code

The `for` loop is the **code**: it knows *how* to play. The arrays are the **data**: *what* to play. To play a different song you only change the data!

Arrays must match

Both arrays need the same number of items, and the `14` in the `for` loop must match. If you add notes, count them carefully.

Name that tune

What song is it? Can you add the next line of the song?

---

5. Build a 5-button keyboard

Put 5 buttons in a row across the middle gap, like piano keys. Each button connects one ESP32 pin to GND, just like in Lesson 3. Keep the passive buzzer on pin 33.

✓	Button	ESP32 pin	Diagonal leg to	Note
☐	1	**13**	– rail	C
☐	2	**14**	– rail	D
☐	3	**16**	– rail	E
☐	4	**17**	– rail	F
☐	5	**18**	– rail	G

Running out of room?

Take the RGB LED and the potentiometer off the breadboard for now. Use short wires and leave one empty row between buttons.

6. Sketch 3: Button piano

File → New Sketch, save it as **L06_Piano**.

Type this in → save it as **L06_Piano** 34 lines

```
// Lesson 6: Button piano
// Passive buzzer on 33. Five buttons on 13, 14, 16, 17, 18
// (other legs to GND) play C, D, E, F, G.

const int BUZZER = 33;

int keys[] = {13, 14, 16, 17, 18};
int freqs[] = {262, 294, 330, 349, 392};
int lastNote = 0;

void setup() {
  for (int i = 0; i < 5; i++) {
    pinMode(keys[i], INPUT_PULLUP);
  }
}

void loop() {
  int note = 0;
  for (int i = 0; i < 5; i++) {
    if (digitalRead(keys[i]) == LOW) {
      note = freqs[i];
    }
  }

  if (note != lastNote) {
    if (note > 0) {
      tone(BUZZER, note);
    } else {
      noTone(BUZZER);
    }
    lastNote = note;
  }
  delay(10);
}
```

Code	What it means
`int keys[] = { … };`	Which pin each key is on
`int freqs[] = { … };`	The note for each key
`int note = 0;`	0 means "no key is pressed"
`if (note != lastNote)`	Only change the sound when a different key is pressed (or let go)

Play it

Try *Mary Had a Little Lamb*: **E D C D E E E**, **D D D**, **E G G**.

7. Challenges

Challenge 1: Your own song

Change the arrays in **L06_Melody** to play a different song. Search for the notes of a song you like, or compose your own!

Challenge 2: Police siren

Make the pitch slide up and down like a siren. *Hint: use a* `for` *loop that counts the frequency from 500 to 1200, calling* `tone()` *with each value.*

Challenge 3: Light-up piano

Put the RGB LED back on pins 25, 26 and 27 (Lesson 5). Make each key light up a different colour while it plays.

Bonus: Knob pitch

Put the potentiometer back on pin 34 and use `map()` so turning the knob changes the pitch, like a theremin.

8. Quiz

Check what you learned

1. Which buzzer can play different notes: active or passive?

   _____

2. What does `tone(BUZZER, 262);` do? How do you stop it?

   _____

   _____

3. If an array has 14 items, what number is the last item?

   _____

Tidy up: unplug the USB cable. Put both buzzers and the buttons back in the kit, and leave the breadboard clear for Lesson 7.

LESSON 7

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Number Display

Light up numbers with a 7-segment display and build an electronic dice.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **7-segment display** makes numbers out of 7 little lights
- How to use a **2D array**, a table of numbers in your code
- How to pick **random** numbers
- How to wait for a button to be **let go**

SESSION PLAN

0:00	How does a 7-segment display work?
0:10	Wire up the display (take your time!)
0:45	Sketch 1: **L07_Segments** (test every light)
0:55	Short break
1:00	Sketch 2: **L07_Count** (numbers 0 to 9)
1:20	Sketch 3: **L07_Dice** (electronic dice)
1:40	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, on your 2 breadboards
- ☐ 1-digit 7-segment display
- ☐ 8 resistors, 220 Ω (red-red-brown)
- ☐ 1 push button
- ☐ About 12 jumper wires

1. Seven little lights

Look closely at the display. The big **8** is really **7 separate LEDs** shaped like bars. There is also a tiny dot, the **decimal point**. Each bar has a letter name, from **a** to **g**:

```
  aaaa
 f    b
 f    b
  gggg
 e    c
 e    c
  dddd   dp
```

Switch on the right bars and you get any number. For a **1**, light **b** and **c**. For an **8**, light all seven.

> **Big idea: one shared leg**
>
> Each of the 8 LEDs has its own **+** leg, but they all share a single **–** leg called the **common cathode** (COM). We connect COM to GND once. Then the ESP32 switches each bar on by setting its pin **HIGH**.

The display has 10 pins: 5 on the top and 5 on the bottom. Hold it with the **decimal point at the bottom right**. Pin 1 is then at the bottom left:

```
   pin: 10  9  8  7  6
        g   f COM a  b       (top row)
          +--------+
          |   8.   |
          +--------+
        e   d COM c  dp      (bottom row)
   pin: 1   2  3  4  5
```

2. Wire up the display

Push the display across the **middle gap** of a free part of a breadboard, with the decimal point at the bottom right. Every bar needs its own **220 Ω** resistor, just like a normal LED.

> **Neat wiring trick**
>
> Put each resistor from the display pin's row across to an empty row. Then run a jumper wire from that row to the ESP32. Check off each row as you go, because it's easy to lose your place!

✓	ESP32 pin	goes to	Display pin	Bar
☐	**23**	220 Ω resistor →	pin 7	a
☐	**22**	220 Ω resistor →	pin 6	b
☐	**21**	220 Ω resistor →	pin 4	c
☐	**19**	220 Ω resistor →	pin 2	d
☐	**18**	220 Ω resistor →	pin 1	e
☐	**17**	220 Ω resistor →	pin 9	f
☐	**16**	220 Ω resistor →	pin 10	g
☐	**4**	220 Ω resistor →	pin 5	dp
☐	**GND**	jumper wire →	pin 3 *or* pin 8	COM

Add the **button** for later: one leg to ESP32 pin **13**, the other leg to the **–** rail (GND).

✓	From	To
☐	button leg	ESP32 pin **13**
☐	other button leg (diagonally opposite)	**–** rail

3. Sketch 1: Test every segment

Before writing numbers, check that every bar works. Type this in and save it as **L07_Segments**.

Type this in → save it as **L07_Segments** 21 lines

```
// Lesson 7: Test every segment
// Lights segments a, b, c, d, e, f, g and dp one at a time.

const int segPins[8] = {23, 22, 21, 19, 18, 17, 16, 4};
const String names[8] = {"a", "b", "c", "d", "e", "f", "g", "dp"};

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 8; i++) {
    pinMode(segPins[i], OUTPUT);
  }
}

void loop() {
  for (int i = 0; i < 8; i++) {
    Serial.println(names[i]);
    digitalWrite(segPins[i], HIGH);
    delay(500);
    digitalWrite(segPins[i], LOW);
  }
}
```

Open the **Serial Monitor** (115200 baud). It tells you which bar *should* be lit. If the letter on screen doesn't match the bar that lights up, two wires are swapped.

Code	What it means
`const int segPins[8] = {23, 22, …};`	An **array**: a list of 8 pin numbers. `segPins[0]` is 23, `segPins[1]` is 22 …
`const String names[8] = {"a", …};`	A list of **words** (strings). `names[0]` is `"a"`
`for (int i = 0; i < 8; i++)`	Repeat for i = 0, 1, 2 … 7: one time for each bar

One bar stays dark?

Find its row in the wiring table and check that wire and resistor. If **nothing** lights, check the COM pin is connected to GND.

4. Sketch 2: Count from 0 to 9

Now we need a **table** that says which bars are on for each number. In code, a table is a **2D array**: an array of rows. Each row has 7 numbers, one for each bar from a to g. `1` means on and `0` means off.

File → New Sketch, type this in, and save it as **L07_Count**.

Type this in → save it as **L07_Count** 37 lines

```
// Lesson 7: Count from 0 to 9
// Each row of the table says which segments are on for one digit.

const int segPins[7] = {23, 22, 21, 19, 18, 17, 16};

const int digits[10][7] = {
// a  b  c  d  e  f  g
  {1, 1, 1, 1, 1, 1, 0},  // 0
  {0, 1, 1, 0, 0, 0, 0},  // 1
  {1, 1, 0, 1, 1, 0, 1},  // 2
  {1, 1, 1, 1, 0, 0, 1},  // 3
  {0, 1, 1, 0, 0, 1, 1},  // 4
  {1, 0, 1, 1, 0, 1, 1},  // 5
  {1, 0, 1, 1, 1, 1, 1},  // 6
  {1, 1, 1, 0, 0, 0, 0},  // 7
  {1, 1, 1, 1, 1, 1, 1},  // 8
  {1, 1, 1, 1, 0, 1, 1}   // 9
};

void showDigit(int n) {
  for (int s = 0; s < 7; s++) {
    digitalWrite(segPins[s], digits[n][s]);
  }
}

void setup() {
  for (int s = 0; s < 7; s++) {
    pinMode(segPins[s], OUTPUT);
  }
}

void loop() {
  for (int n = 0; n <= 9; n++) {
    showDigit(n);
    delay(1000);
  }
}
```

Code	What it means
`const int digits[10][7]`	A table with **10 rows** (digits 0–9) and **7 columns** (bars a–g)
`digits[n][s]`	Row **n**, column **s**. For example `digits[1][0]` is 0: bar **a** is off for the digit 1
`void showDigit(int n)`	Our own **function**. Give it a number and it switches on the right bars
`digitalWrite(segPins[s], digits[n][s]);`	`1` works just like `HIGH`, and `0` works just like `LOW`

Try it

1. Look at the row for **7**. Which bars are on? Check it on the display.
2. Change `delay(1000)` to `delay(200)`. Super fast counting!
3. Change the loop so it only shows **even** numbers: 0, 2, 4, 6, 8. *Hint:* `n = n + 2`.

5. Sketch 3: Electronic dice

Time for a game! When you press the button, the display "rolls" through random numbers, slowing down, then stops on a number from 1 to 6.

File → Save As… your L07_Count sketch with the new name **L07_Dice**. Then change it to match this code. The big table stays the same, so you only need to type the new parts!

Type this in → save it as **L07_Dice** — 50 lines

```
// Lesson 7: Electronic dice
// Press the button on pin 13 to roll.

const int BUTTON_PIN = 13;
const int segPins[7] = {23, 22, 21, 19, 18, 17, 16};

const int digits[10][7] = {
// a  b  c  d  e  f  g
  {1, 1, 1, 1, 1, 1, 0},  // 0
  {0, 1, 1, 0, 0, 0, 0},  // 1
  {1, 1, 0, 1, 1, 0, 1},  // 2
  {1, 1, 1, 1, 0, 0, 1},  // 3
  {0, 1, 1, 0, 0, 1, 1},  // 4
  {1, 0, 1, 1, 0, 1, 1},  // 5
  {1, 0, 1, 1, 1, 1, 1},  // 6
  {1, 1, 1, 0, 0, 0, 0},  // 7
  {1, 1, 1, 1, 1, 1, 1},  // 8
  {1, 1, 1, 1, 0, 1, 1}   // 9
};

void showDigit(int n) {
  for (int s = 0; s < 7; s++) {
    digitalWrite(segPins[s], digits[n][s]);
```

```
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  for (int s = 0; s < 7; s++) {
    pinMode(segPins[s], OUTPUT);
  }
  showDigit(0);
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {
    for (int i = 0; i < 12; i++) {  // rolling animation
      showDigit(random(1, 7));
      delay(40 + i * 15);
    }
    int roll = random(1, 7);
    showDigit(roll);
    Serial.print("You rolled a ");
    Serial.println(roll);
    while (digitalRead(BUTTON_PIN) == LOW) {
      delay(10);  // wait until the button is let go
    }
  }
}
```

Code	What it means
`random(1, 7)`	A random whole number from **1 up to 6**. The 7 itself is never chosen!
`delay(40 + i * 15);`	Each step waits a bit longer, so the roll **slows down** like a real dice
`while (digitalRead(BUTTON_PIN) == LOW)`	Keep waiting **while** the button is still held down

Big idea: really random

The ESP32 has a special circuit inside that makes **truly random** numbers from electrical noise. Most small microcontrollers can only fake it!

Try it

1. Roll the dice 20 times. Write down how many times you get a **6**.

---

2. What happens if you **delete** the `while` loop and hold the button down?

6. Challenges

Challenge 1: Countdown

Make the display count **down** from 9 to 0, then flash the 0 three times like a rocket launch. *Hint:* `for (int n = 9; n >= 0; n--)`.

Challenge 2: Letters!

A 7-segment display can also show **A b C d E F**. Add 6 more rows to the table (make it `[16][7]`), then count from 0 to F. Computer scientists call this **hexadecimal**.

Challenge 3: Dice with sound

Connect a **passive buzzer** (+ to pin **33**, – to GND). Make a click with `tone(33, 1500, 10);` on every step of the roll, and a happy beep at the end.

Bonus: Spinner

While the dice rolls, light just one outside bar at a time (a, b, c, d, e, f) so a light chases around the edge.

7. Quiz

Check what you learned

1. Which bars light up to make the number **4**?

   ---

2. In `digits[3][6]`, which row and which column is that?

   ---

3. What numbers can `random(1, 7)` give you?

Tidy up: unplug the USB cable. Leave the display and its resistors in place, because Lesson 9 uses the same wiring for the 4-digit display!

LESSON 8

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Shift Register Magic

Control 8 LEDs with only 3 wires, and learn to count like a computer.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **shift register** (74HC595) gives you extra outputs
- How computers count in **binary** with only 0 and 1
- What a **bit** and a **byte** are
- How to store a light show in an **array** of patterns

SESSION PLAN

0:00	What is a shift register? What is binary?
0:15	Wire the chip and 8 LEDs
0:50	Sketch 1: **L08_OneByOne**
0:55	Short break
1:00	Sketch 2: **L08_Binary** (count like a computer)
1:20	Sketch 3: **L08_LightShow**
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, on your 2 breadboards
- ☐ 74HC595 chip (16 legs)
- ☐ 8 LEDs (any colours)
- ☐ 8 resistors, 220 Ω (red-red-brown)
- ☐ 10 kΩ potentiometer (the knob)
- ☐ About 20 jumper wires

1. Running out of pins?

Last lesson, one small display used up **8 pins**. What if you want 16 LEDs, or 32? The ESP32 would quickly run out of pins.

The **74HC595 shift register** is a little chip that fixes this. The ESP32 sends it 8 on/off values **one after another** down a single data wire. The chip collects them, then switches its 8 outputs all at once. That's **3 wires in, 8 outputs out**.

Big idea: three wires

- **DATA**: the next value, on (1) or off (0).
- **CLOCK**: a "tick" that says *take the next value now*. Each tick pushes (shifts) all the values one place along.
- **LATCH**: "Done! Show all 8 values on the outputs now."

2. Binary: counting with only 0 and 1

We count with ten digits (0–9). Computers only have **off** and **on**, so they count with two digits: **0** and **1**. This is called **binary**. One 0-or-1 is called a **bit**, and 8 bits make a **byte**.

Each place in a binary number is worth double the place to its right:

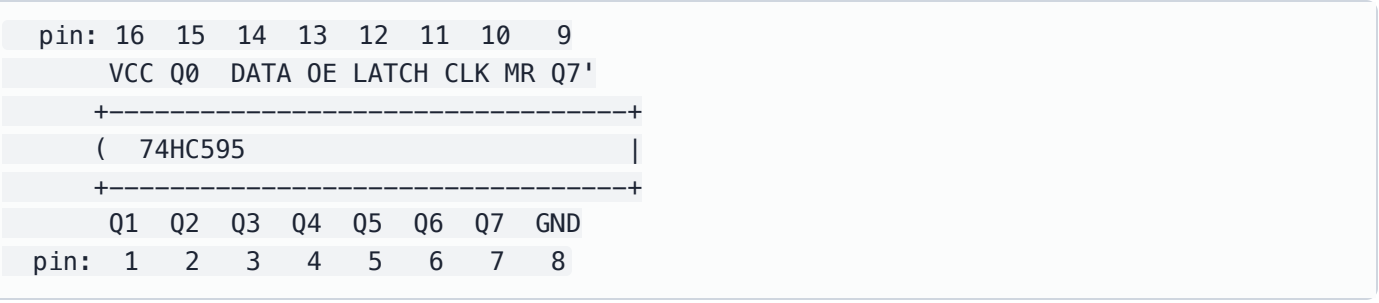
Place value	128	64	32	16	8	4	2	1
5 in binary	0	0	0	0	0	1	0	1
12 in binary	0	0	0	0	1	1	0	0
255 in binary	1	1	1	1	1	1	1	1

5 = 4 + 1, and 12 = 8 + 4. In code we write binary with **0b** in front: `0b00000101` is 5.

> **Try it on paper**
>
> Write **your age** as 8 bits:
>
> _____

3. Wire it up

The chip has a little **half-moon notch** at one end. Push the chip across the middle gap of a breadboard with the **notch pointing left**. Then pin 1 is bottom-left:



First give the chip power and connect it to the ESP32:

✓	From	To
☐	ESP32 **3V3**	**+** rail
☐	ESP32 **GND**	**–** rail
☐	chip pin **16** (VCC)	**+** rail
☐	chip pin **10** (MR)	**+** rail
☐	chip pin **8** (GND)	**–** rail
☐	chip pin **13** (OE)	**–** rail
☐	chip pin **14** (DATA)	ESP32 pin **23**
☐	chip pin **12** (LATCH)	ESP32 pin **19**
☐	chip pin **11** (CLOCK)	ESP32 pin **18**

Now put the 8 LEDs in a line on the breadboard. Each LED's **short leg** goes to a **220 Ω resistor**, and the resistor goes to the **–** rail. Then connect each **long leg** to a chip output. Put **Q7 on the left** and **Q0 on the right**, so the LEDs read like a binary number.

✓	LED (left → right)	long leg goes to chip pin
☐	LED 1 (128)	pin **7** (Q7)
☐	LED 2 (64)	pin **6** (Q6)
☐	LED 3 (32)	pin **5** (Q5)
☐	LED 4 (16)	pin **4** (Q4)
☐	LED 5 (8)	pin **3** (Q3)
☐	LED 6 (4)	pin **2** (Q2)
☐	LED 7 (2)	pin **1** (Q1)
☐	LED 8 (1)	pin **15** (Q0)

Last, add the **potentiometer** (the knob). We'll use it in Sketch 3:

✓	From	To
☐	potentiometer left leg	**+** rail (3V3)
☐	potentiometer middle leg	ESP32 pin **34**
☐	potentiometer right leg	**–** rail (GND)

Careful with the chip

Don't connect pin 16 to 5V: use **3V3**. If the chip gets hot, unplug the USB right away and check the power wires.

4. Sketch 1: One LED at a time

Type this in and save it as **L08_OneByOne**. You should see a light walk from right to left, then the LEDs flash every other one.

Lights in the wrong order?

If the light walks the wrong way or jumps around, check the LED wiring table again. If nothing lights, check pins 10 and 16 go to **+** and pins 8 and 13 go to **–**.

Type this in → save it as **L08_OneByOne** 31 lines

```
// Lesson 8: One LED at a time
// 74HC595 shift register: DATA 23, LATCH 19, CLOCK 18.

const int DATA_PIN = 23;
const int LATCH_PIN = 19;
const int CLOCK_PIN = 18;

void showByte(byte leds) {
  digitalWrite(LATCH_PIN, LOW);
  shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, leds);
  digitalWrite(LATCH_PIN, HIGH);
}
```

```
void setup() {
  pinMode(DATA_PIN, OUTPUT);
  pinMode(LATCH_PIN, OUTPUT);
  pinMode(CLOCK_PIN, OUTPUT);
}

void loop() {
  for (int i = 0; i < 8; i++) {
    showByte(bit(i));
    delay(200);
  }
  for (int i = 0; i < 4; i++) {
    showByte(0b10101010);
    delay(300);
    showByte(0b01010101);
    delay(300);
  }
}
```

Code	What it means
`byte leds`	A **byte** is a number from 0 to 255: exactly 8 bits, one for each LED
`digitalWrite(LATCH_PIN, LOW);`	"Get ready, I'm sending new values"
`shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, leds);`	Send the 8 bits one at a time, ticking the clock for each. **MSBFIRST** means the left-most bit goes first
`digitalWrite(LATCH_PIN, HIGH);`	"Show them now!" All 8 LEDs change at once
`bit(i)`	A byte with only bit **i** switched on. `bit(3)` is `0b00001000`
`0b10101010`	A binary number: every other LED is on

5. Sketch 2: Count in binary

Save a copy as **L08_Binary** and change it to match this. Open the Serial Monitor too.

Type this in → save it as **L08_Binary** — 29 lines

```
// Lesson 8: Count in binary
// The 8 LEDs show the number in binary. Watch the Serial Monitor too!

const int DATA_PIN = 23;
const int LATCH_PIN = 19;
const int CLOCK_PIN = 18;

void showByte(byte leds) {
  digitalWrite(LATCH_PIN, LOW);
  shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, leds);
  digitalWrite(LATCH_PIN, HIGH);
```

```
}

void setup() {
  Serial.begin(115200);
  pinMode(DATA_PIN, OUTPUT);
  pinMode(LATCH_PIN, OUTPUT);
  pinMode(CLOCK_PIN, OUTPUT);
}

void loop() {
  for (int n = 0; n < 256; n++) {
    showByte(n);
    Serial.print(n);
    Serial.print(" in binary is ");
    Serial.println(n, BIN);
    delay(500);
  }
}
```

Code	What it means
`showByte(n);`	Show the number **n** on the LEDs as binary
`Serial.println(n, BIN);`	Print **n** in binary instead of normal numbers

Try it

1. Watch the right-most LED. How often does it change?
2. When does the **left-most** LED switch on for the first time? Which number is it?

---

3. Change `delay(500)` to `delay(20)`. How long does it take to count to 255 now?

6. Sketch 3: Light show

Now we'll store a whole light show in an **array** of patterns, and use the knob to change the speed. Save a copy as **L08_LightShow** and change it to match:

Type this in → save it as **L08_LightShow** — 39 lines

```
// Lesson 8: Light show
// Plays a list of patterns. The knob on pin 34 sets the speed.

const int DATA_PIN = 23;
const int LATCH_PIN = 19;
const int CLOCK_PIN = 18;
const int POT_PIN = 34;

const int NUM_PATTERNS = 8;
const byte patterns[NUM_PATTERNS] = {
  0b10000001,
```

```
  0b01000010,
  0b00100100,
  0b00011000,
  0b00011000,
  0b00100100,
  0b01000010,
  0b10000001
};

void showByte(byte leds) {
  digitalWrite(LATCH_PIN, LOW);
  shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, leds);
  digitalWrite(LATCH_PIN, HIGH);
}

void setup() {
  pinMode(DATA_PIN, OUTPUT);
  pinMode(LATCH_PIN, OUTPUT);
  pinMode(CLOCK_PIN, OUTPUT);
}

void loop() {
  for (int i = 0; i < NUM_PATTERNS; i++) {
    showByte(patterns[i]);
    int wait = map(analogRead(POT_PIN), 0, 4095, 30, 500);
    delay(wait);
  }
}
```

Code	What it means
`const byte patterns[NUM_PATTERNS] = {…};`	A list of 8 patterns. Each line is one "frame" of the show
`map(analogRead(POT_PIN), 0, 4095, 30, 500)`	Turn the knob reading (0–4095) into a wait of 30–500 ms

Design on paper first

Draw 8 circles for each frame, colour in the ones that should be on, then write the 1s and 0s.

7. Challenges

Challenge 1: Your own show

Change the patterns to make your own light show. Try to make it at least 12 frames long. Remember to change `NUM_PATTERNS`!

Challenge 2: Knight Rider

Make one light sweep from right to left and back again, like a scanner. *Hint: use two* `for` *loops, one going up and one going down, with* `bit(i)`.

Challenge 3: Bar graph

Make a meter. The further you turn the knob, the more LEDs light up, from 0 to 8. *Hint:* `map(…, 0, 4095, 0, 8)` *tells you how many to light.*

Bonus: Binary quiz

Show a random number from 1 to 15 on the LEDs. Your friend has to say the number in normal counting!

8. Quiz

Check what you learned

1. What do the three wires DATA, CLOCK and LATCH do?

   _____

   _____

2. Write the number **10** in binary (8 bits).

   _____

3. What is the biggest number one byte can hold?

   _____

Tidy up: unplug the USB cable. Gently lever the chip out with a flat tool from both ends so you don't bend its legs.

LESSON 9

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Reaction Timer

Drive a 4-digit display with a clever trick and test how fast you are.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How **multiplexing** fools your eyes into seeing 4 digits at once
- How to use **millis()** as a stopwatch instead of `delay()`
- How to split a number into digits with **/** and **%**
- How to build a game with **while** loops

SESSION PLAN

0:00	How can 4 digits share the same wires?
0:10	Wire up the 4-digit display
0:40	Sketch 1: **L09_Multiplex**
0:55	Short break
1:00	Sketch 2: **L09_Counter** (millis and digits)
1:20	Sketch 3: **L09_Reaction** (the game!)
1:40	Challenges: beat your friends, catch cheaters
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, on your 2 breadboards
- ☐ 4-digit 7-segment display (12 pins)
- ☐ 8 resistors, 220 Ω (red-red-brown)
- ☐ 1 push button
- ☐ About 16 jumper wires

1. Four digits, one trick

The 4-digit display has **4 × 8 = 32** little LEDs, but only **12 pins**. How?

All four digits **share** the same 8 segment wires (a–g and dp). Each digit also has its own **digit pin**, which is its shared **–** leg. A digit only lights up when its digit pin is **LOW** (connected to 0 V).

So the ESP32 does this, over and over:

1. Switch on the bars for the **1st** digit, light only digit 1, then wait a tiny moment
2. Switch on the bars for the **2nd** digit, light only digit 2, then wait a tiny moment
3. … digit 3 … digit 4 … then back to the start

> **Big idea: multiplexing**
>
> Only **one** digit is ever lit at a time! But the ESP32 switches between them about **100 times a second**. That's too fast for your eyes, so you see all four at once. Films and TV screens fool your eyes in a similar way.

Hold the display with the **decimal points at the bottom**. Pin 1 is at the bottom left:

```
  pin: 12  11  10   9   8   7
       D1   a   f  D2  D3   b       (top row)
      +-------------------------+
      |   8.   8.   8.   8.     |
      +-------------------------+
       e   d  dp   c   g  D4       (bottom row)
  pin: 1   2   3   4   5   6
```

2. Wire it up

Push the display across the middle gap of a breadboard. You can reuse the ESP32 wires from Lesson 7: the **segment pins on the ESP32 are the same**. Only the display's pin positions are different.

Segments: each one goes through a 220 Ω resistor.

✓	ESP32 pin	goes to	Display pin	Segment
☐	**23**	220 Ω resistor →	pin 11	a
☐	**22**	220 Ω resistor →	pin 7	b
☐	**21**	220 Ω resistor →	pin 4	c
☐	**19**	220 Ω resistor →	pin 2	d
☐	**18**	220 Ω resistor →	pin 1	e
☐	**17**	220 Ω resistor →	pin 10	f
☐	**16**	220 Ω resistor →	pin 5	g
☐	**4**	220 Ω resistor →	pin 3	dp

Digits: these connect straight to the ESP32 with a jumper wire, *without* resistors.

✓	ESP32 pin	Display pin	Digit
☐	**25**	pin 12	D1 (left)
☐	**26**	pin 9	D2
☐	**27**	pin 8	D3
☐	**32**	pin 6	D4 (right)

Button: one leg to ESP32 pin **13**, the diagonally opposite leg to the **–** rail. Connect ESP32 **GND** to the **–** rail.

3. Sketch 1: See the trick

Type this in and save it as **L09_Multiplex**. The digit table is the same as in Lesson 7. If you still have L07_Count, you can copy and paste it!

Type this in → save it as **L09_Multiplex** 49 lines

```
// Lesson 9: Four digits, one at a time
// Shows 1234 by lighting each digit in turn, very fast.

const int segPins[7] = {23, 22, 21, 19, 18, 17, 16};
const int digitPins[4] = {25, 26, 27, 32};

const int digits[10][7] = {
// a  b  c  d  e  f  g
  {1, 1, 1, 1, 1, 1, 0},  // 0
  {0, 1, 1, 0, 0, 0, 0},  // 1
  {1, 1, 0, 1, 1, 0, 1},  // 2
  {1, 1, 1, 1, 0, 0, 1},  // 3
  {0, 1, 1, 0, 0, 1, 1},  // 4
  {1, 0, 1, 1, 0, 1, 1},  // 5
  {1, 0, 1, 1, 1, 1, 1},  // 6
  {1, 1, 1, 0, 0, 0, 0},  // 7
  {1, 1, 1, 1, 1, 1, 1},  // 8
  {1, 1, 1, 1, 0, 1, 1}   // 9
};

void showDigit(int place, int n) {
  for (int d = 0; d < 4; d++) {
    digitalWrite(digitPins[d], HIGH);  // all digits off
  }
  for (int s = 0; s < 7; s++) {
    digitalWrite(segPins[s], digits[n][s]);
  }
  digitalWrite(digitPins[place], LOW);  // this digit on
}

void setup() {
  for (int s = 0; s < 7; s++) {
    pinMode(segPins[s], OUTPUT);
  }
  for (int d = 0; d < 4; d++) {
    pinMode(digitPins[d], OUTPUT);
  }
}

void loop() {
  showDigit(0, 1);
```

```
  delay(5);
  showDigit(1, 2);
  delay(5);
  showDigit(2, 3);
  delay(5);
  showDigit(3, 4);
  delay(5);
}
```

You should see **1234**.

Code	What it means
`const int digitPins[4]`	The 4 pins that choose **which digit** is lit
`void showDigit(int place, int n)`	Show number **n** at position **place** (0 = left, 3 = right)
`digitalWrite(digitPins[d], HIGH);`	**HIGH** switches a digit **off**, because its – leg isn't at 0 V any more
`digitalWrite(digitPins[place], LOW);`	**LOW** switches just this one digit **on**

Try it: catch the trick!

1. Change all four `delay(5)` to `delay(200)`. Now you can see what is really happening!
2. Try `delay(20)`. Does it flicker? Find the biggest delay that still looks steady to you:

   _____
3. Make it show your birth year, or **4321**.

Some segments look dim or wrong?

Check the segment table. Remember, the segment pins on this display are in **different places** from the Lesson 7 display. If one whole digit is dark, check its digit wire.

4. Sketch 2: A seconds counter

To show a number like **2735**, we must split it into four digits. Two maths symbols help:

Symbol	Meaning	Example
`/`	Divide, and **throw away** the remainder	`2735 / 100` is **27**
`%`	Just the **remainder** after dividing	`27 % 10` is **7**

So `2735 / 1000 % 10` is 2, `2735 / 100 % 10` is 7, `2735 / 10 % 10` is 3, and `2735 / 1 % 10` is 5.

File → Save As… your L09_Multiplex sketch with the new name **L09_Counter**. Then make two changes. Everything else stays the same!

Change 1: add this new function just **below** `showDigit`:

```
void showNumber(int number) {
  number = constrain(number, 0, 9999);
  int divisor = 1000;
  for (int place = 0; place < 4; place++) {
    showDigit(place, number / divisor % 10);
    delay(2);
    divisor = divisor / 10;
  }
}
```

Change 2: replace the whole `loop()` with this much shorter one:

```
void loop() {
  int seconds = millis() / 1000;
  showNumber(seconds);
}
```

Code	What it means
`millis()`	How many **milliseconds** since the ESP32 started. It's like a stopwatch that never stops
`constrain(number, 0, 9999)`	Keep the number between 0 and 9999, because 4 digits can't show more
`divisor = divisor / 10;`	1000 → 100 → 10 → 1: one step for each digit

> **Big idea: why not use delay()?**
>
> `delay(1000)` would freeze the ESP32 for a whole second. Nothing could be shown on the display, so it would go **dark**! With `millis()` the ESP32 just *checks the time* and keeps on refreshing the display.

> **Try it**
>
> Change `millis() / 1000` to `millis() / 100`. What is it counting now? Then press the **EN** button. What happens?

5. Sketch 3: Reaction timer game

How fast are you? The display shows **0** for a random time. Then it starts counting up in milliseconds. Press the button as fast as you can to stop it!

Save a copy as **L09_Reaction** and add the new parts:

Type this in → save it as **L09_Reaction** 69 lines

```
// Lesson 9: Reaction timer game
// When the numbers start counting, press the button as fast as you can!

const int BUTTON_PIN = 13;
const int segPins[7] = {23, 22, 21, 19, 18, 17, 16};
const int digitPins[4] = {25, 26, 27, 32};

const int digits[10][7] = {
// a  b  c  d  e  f  g
  {1, 1, 1, 1, 1, 1, 0},  // 0
  {0, 1, 1, 0, 0, 0, 0},  // 1
  {1, 1, 0, 1, 1, 0, 1},  // 2
  {1, 1, 1, 1, 0, 0, 1},  // 3
  {0, 1, 1, 0, 0, 1, 1},  // 4
  {1, 0, 1, 1, 0, 1, 1},  // 5
  {1, 0, 1, 1, 1, 1, 1},  // 6
  {1, 1, 1, 0, 0, 0, 0},  // 7
  {1, 1, 1, 1, 1, 1, 1},  // 8
  {1, 1, 1, 1, 0, 1, 1}   // 9
};

void showDigit(int place, int n) {
  for (int d = 0; d < 4; d++) {
    digitalWrite(digitPins[d], HIGH);  // all digits off
  }
  for (int s = 0; s < 7; s++) {
    digitalWrite(segPins[s], digits[n][s]);
  }
  digitalWrite(digitPins[place], LOW);  // this digit on
}

void showNumber(int number) {
  number = constrain(number, 0, 9999);
  int divisor = 1000;
  for (int place = 0; place < 4; place++) {
    showDigit(place, number / divisor % 10);
    delay(2);
    divisor = divisor / 10;
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  for (int s = 0; s < 7; s++) {
    pinMode(segPins[s], OUTPUT);
  }
```

```
  for (int d = 0; d < 4; d++) {
    pinMode(digitPins[d], OUTPUT);
  }
}

void loop() {
  unsigned long goTime = millis() + random(2000, 6000);
  while (millis() < goTime) {
    showNumber(0);  // get ready...
  }
  while (digitalRead(BUTTON_PIN) == HIGH) {
    showNumber(millis() - goTime);  // GO! count until pressed
  }
  int score = millis() - goTime;
  Serial.print("Your time: ");
  Serial.print(score);
  Serial.println(" ms");
  unsigned long showUntil = millis() + 3000;
  while (millis() < showUntil) {
    showNumber(score);
  }
}
```

Code	What it means
`unsigned long goTime`	A **long** number for big times. **unsigned** means it is never negative
`millis() + random(2000, 6000)`	"Now, plus 2 to 6 seconds", so nobody knows when it will start
`while (millis() < goTime)`	Keep showing 0 until it's time to go
`while (digitalRead(BUTTON_PIN) == HIGH)`	Keep counting **while** the button is not pressed

6. Challenges

Challenge 1: Catch the cheater

What happens if you hold the button down **before** the counting starts? Fix the game so it shows **9999** for cheaters. *Hint: check the button inside the "get ready" loop.*

Challenge 2: Best score

Make a variable `best` that remembers the fastest time. After each go, print `"New record!"` in the Serial Monitor if you beat it. Then have a competition with your friends: 3 tries each!

Challenge 3: Stopwatch

Make a stopwatch: press once to start, press again to stop, and press a third time to reset. Show tenths of a second, and switch on the **decimal point** to show **12.3**.

7. Quiz

Check what you learned

1. In multiplexing, how many digits are lit at exactly the same moment?

   ---

2. What are `4567 / 100` and `4567 % 10`?

   ---

3. Why does `delay(1000)` make the 4-digit display go dark?

Tidy up: unplug the USB cable. Take everything off the breadboard except the ESP32. Next lesson uses a screen!

LESSON 10

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Tiny Screen

Write words, draw pictures and make animations on an OLED screen.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How to install a **library** (code other people wrote for you)
- How **I²C** lets parts talk over just two wires
- How screen **coordinates** (x, y) work
- How to make an **animation** frame by frame

SESSION PLAN

0:00	What's an OLED? Install the library
0:15	Wire up the screen
0:25	Sketch 1: **L10_Hello** (text and numbers)
0:50	Short break
0:55	Sketch 2: **L10_Shapes** (robot face)
1:20	Sketch 3: **L10_Bounce** (animation)
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, on your 2 breadboards
- ☐ 0.96" OLED display module (4 pins)
- ☐ 4 jumper wires
- ☐ *For the challenges:* 10 kΩ potentiometer + 3 jumper wires

1. A screen for your ESP32

Your kit has a tiny **OLED** screen: 128 dots across and 64 dots down. Each dot is called a **pixel**. "OLED" means every pixel is a tiny light of its own, so it glows brightly on a black background.

Drawing letters dot by dot would be a LOT of work. Luckily, somebody already wrote that code and shared it as a **library**. We just have to install it once.

> **Big idea: libraries**
>
> A **library** is a package of ready-made code. You add it to your sketch with `#include`. Then you can use its commands, like `display.println("Hi")`, without knowing how they work inside.

Install the library (once per computer, and your teacher may have done it already):

- ☐ In the Arduino IDE, open **Tools** → **Manage Libraries…**
- ☐ Search for **Adafruit SSD1306**
- ☐ Click **Install**. When it asks about extra libraries it needs, click **Install All**. (This also installs *Adafruit GFX*, which does the drawing.)

2. Wire up the screen

The screen talks to the ESP32 using **I²C** (say "I-squared-C"). It needs just two signal wires: **SDA** carries the data, and **SCL** is the clock that keeps time. On the ESP32, I²C always uses pins **21** and **22**.

Read the labels printed on **your** screen's pins. They are not always in the same order! Hold the screen by its pins, not the thin glass.

✓	OLED pin	ESP32 pin
☐	**GND**	**GND**
☐	**VCC** (or VDD)	**3V3**
☐	**SCL** (or SCK)	**22**
☐	**SDA**	**21**

3. Sketch 1: Hello, screen!

Type this in and save it as **L10_Hello**:

Type this in → save it as **L10_Hello** — 26 lines

```
// Lesson 10: Hello, screen!
// OLED screen: SDA on pin 21, SCL on pin 22.

#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

int count = 0;

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Hello, I am Sparky!");
  display.setTextSize(3);
  display.setCursor(0, 24);
  display.println(count);
  display.display();
  count = count + 1;
  delay(500);
}
```

You should see a message and a number that counts up.

Screen stays dark?

Check the four wires, especially that SDA and SCL aren't swapped. Some screens use address **0x3D**: try changing `0x3C` to `0x3D`.

Code	What it means
`#include <Adafruit_SSD1306.h>`	Use the screen library
`Adafruit_SSD1306 display(128, 64, &Wire, -1);`	Make a screen called **display**: 128 × 64 pixels, on the I²C wires (`Wire`). The `&` and `-1` are just part of the recipe
`display.begin(SSD1306_SWITCHCAPVCC, 0x3C)`	Switch the screen on. **0x3C** is its I²C **address**, like a house number
`display.clearDisplay();`	Wipe the picture clean
`display.setTextSize(3);`	Make the letters 3 times bigger
`display.setCursor(0, 24);`	Start writing at x = 0, y = 24
`display.display();`	**Send** the picture to the screen. Nothing appears until you do this!

Big idea: draw, then show

The library draws your picture in the ESP32's memory first. When the picture is ready, `display.display()` sends all of it to the screen in one go. That's why the screen never shows half-drawn pictures.

Try it

1. Change `"Hello, I am Sparky!"` to your own message.
2. Try text size **1**, **2**, **4**. How many big letters fit across?

---

4. Sketch 2: Draw with shapes

The screen is like graph paper. **x** counts pixels **across** from the left (0 to 127). **y** counts pixels **down** from the top (0 to 63). Watch out: y goes *down*, not up like in maths class!

```
(0,0) ──────── x ──────────▶ (127,0)
  │
  y        (64,32) is the middle
  │
  ▼
(0,63)                      (127,63)
```

Save as **L10_Shapes**:

Type this in → save it as **L10_Shapes** 34 lines

```
// Lesson 10: Draw with shapes
// A robot face that blinks. x goes 0–127 across, y goes 0–63 down.

#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

void drawEyes(int color) {
  display.fillCircle(48, 30, 7, color);
  display.fillCircle(80, 30, 7, color);
}

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.clearDisplay();
  display.drawRoundRect(24, 10, 80, 53, 8, SSD1306_WHITE);  // head
  display.drawLine(64, 10, 64, 3, SSD1306_WHITE);           // antenna
  display.fillCircle(64, 3, 3, SSD1306_WHITE);
  display.fillTriangle(64, 36, 59, 44, 69, 44, SSD1306_WHITE);
  display.drawRect(46, 50, 36, 7, SSD1306_WHITE);           // mouth
  drawEyes(SSD1306_WHITE);
  display.display();
}

void loop() {
  delay(3000);
  drawEyes(SSD1306_BLACK);  // close the eyes
  display.drawLine(41, 30, 55, 30, SSD1306_WHITE);
  display.drawLine(73, 30, 87, 30, SSD1306_WHITE);
  display.display();
  delay(200);
  drawEyes(SSD1306_WHITE);  // open them again
  display.display();
}
```

Command	Draws
`drawLine(x1, y1, x2, y2, color)`	A line from one point to another
`drawRect(x, y, width, height, color)`	A rectangle outline. `fillRect` draws a solid one
`drawRoundRect(x, y, w, h, radius, color)`	A rectangle with rounded corners
`fillCircle(x, y, radius, color)`	A solid circle centred on (x, y). `drawCircle` draws just the outline
`fillTriangle(x1, y1, x2, y2, x3, y3, color)`	A solid triangle with 3 corners
`SSD1306_BLACK`	Drawing in black **rubs out**. That's how the eyes close

Plan on paper

Draw your picture on squared paper first and write down the numbers for each corner.

5. Sketch 3: Bouncing ball

An animation is just lots of pictures shown quickly, one after another. Each picture is called a **frame**. Every frame we: move the ball a little → wipe the screen → draw the ball → show it.

Save as **L10_Bounce**:

Type this in → save it as **L10_Bounce** — 32 lines

```
// Lesson 10: Bouncing ball
// The ball moves a little every frame and bounces off the edges.

#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

const int R = 4;  // ball radius
int x = 20;
int y = 10;
int dx = 2;  // how far it moves across each frame
int dy = 1;  // how far it moves down each frame

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
  x = x + dx;
  y = y + dy;
  if (x <= R || x >= 127 - R) {
    dx = -dx;
  }
  if (y <= R || y >= 63 - R) {
    dy = -dy;
  }
  display.clearDisplay();
  display.drawRect(0, 0, 128, 64, SSD1306_WHITE);
  display.fillCircle(x, y, R, SSD1306_WHITE);
  display.display();
  delay(10);
}
```

Code	What it means
`x = x + dx;`	Move the ball **dx** pixels across (and `y + dy` moves it down)
`x <= R || x >= 127 - R`	`||` means **or**: the ball is touching the left edge *or* the right edge
`dx = -dx;`	Turn around! 2 becomes –2 and –2 becomes 2

Try it

1. Make the ball **bigger** by changing `R`.
2. Make it **faster**. Which two numbers do you change?
3. Delete `display.clearDisplay();`. What happens, and why?

---

6. Challenges

Challenge 1: Your own picture

Change L10_Shapes to draw something of your own: a house, a cat, a rocket, or your initials in shapes.

Challenge 2: Knob meter

Connect a potentiometer (outer legs to 3V3 and GND, middle leg to pin **34**). Show the knob position as a number from 0 to 100% **and** as a bar that grows. *Hint:* `map(analogRead(34), 0, 4095, 0, 100)` *and* `fillRect`.

Challenge 3: Two balls

Add a second ball with its own `x2`, `y2`, `dx2`, `dy2` that bounces at a different speed.

Bonus: Paddle game

Draw a paddle along the bottom that you move with the knob. If the ball hits the paddle it bounces up and you score a point. If it misses, the score goes back to 0!

7. Quiz

Check what you learned

1. What does a library do for you?

---

2. Which pixel is at the **bottom-right** corner of the screen?

---

3. Why does nothing show up if you forget `display.display();`?

Tidy up: unplug the USB cable. Leave the screen wired up, because Lesson 11 uses it to build a weather station!

LESSON 11

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Weather Station

Measure temperature and humidity and show them on your screen.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **sensor** turns the world into numbers
- What **humidity** is
- How to use **float** numbers with a decimal point
- How to make decisions with **if / else if**

SESSION PLAN

0:00	How do sensors work? Install the DHT library
0:15	Wire up the DHT11
0:25	Sketch 1: **L11_ReadDHT** (Serial Monitor)
0:50	Short break
0:55	Sketch 2: **L11_Station** (on the screen)
1:15	Sketch 3: **L11_Comfort** (comfort face)
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, on your 2 breadboards
- ☐ OLED screen, still wired from Lesson 10
- ☐ DHT11 temperature and humidity module (blue, 3 pins)
- ☐ 3 jumper wires
- ☐ *For a challenge:* thermistor + 10 kΩ resistor (brown-black-orange)

1. Measuring the weather

A **sensor** is a part that measures something in the world and turns it into an electrical signal. Your microcontroller can then read it as a number.

The blue **DHT11** module has two sensors in one:

- a **thermometer** that measures **temperature** in degrees Celsius (°C)
- a **humidity** sensor that measures how much **water vapour** is in the air, in percent (%)

> **Big idea: humidity**
>
> Air always holds some invisible water. 30% humidity feels dry (your lips get chapped). 80% feels damp and sticky, like a bathroom after a hot shower. Most people feel comfortable between about 30% and 60%.

The DHT11 has a tiny chip inside that does the measuring. It sends the answers to the ESP32 as a stream of 1s and 0s on **one wire**. Decoding that stream is tricky, so we use a library.

Install the library:

- ☐ **Tools → Manage Libraries…**
- ☐ Search for **DHT sensor library** (by Adafruit)
- ☐ Click **Install**, then **Install All** to get *Adafruit Unified Sensor* too.

2. Wire up the DHT11

Keep the OLED screen wired up from Lesson 10. Push the DHT11 module into the breadboard and **read the labels** on its pins. They are usually **S** (signal), **+** and **–**.

✓	DHT11 pin	ESP32 pin
☐	**S** (signal / DATA / OUT)	**4**
☐	**+** (VCC)	**3V3**
☐	**–** (GND)	**GND**

> **Check before you plug in**
>
> If + and – are swapped, the sensor gets hot and can be damaged. Double-check, then plug in the USB cable.

3. Sketch 1: Read the sensor

Type this in and save it as **L11_ReadDHT**. Open the **Serial Monitor** (115200 baud).

Type this in → save it as **L11_ReadDHT** — 27 lines

```
// Lesson 11: Read temperature and humidity
// DHT11 sensor: data pin on pin 4.

#include <DHT.h>

DHT dht(4, DHT11);

void setup() {
  Serial.begin(115200);
  dht.begin();
}

void loop() {
  delay(2000);  // the DHT11 needs 2 seconds between readings
  float temp = dht.readTemperature();
  float humidity = dht.readHumidity();

  if (isnan(temp) || isnan(humidity)) {
    Serial.println("Could not read the sensor!");
    return;
  }
  Serial.print("Temperature: ");
  Serial.print(temp);
  Serial.print(" C   Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");
}
```

Code	What it means
`#include <DHT.h>`	Use the DHT library
`DHT dht(4, DHT11);`	Make a sensor called **dht**, on pin 4, of type **DHT11**
`float temp`	A **float** is a number that can have a decimal point, like 23.5
`dht.readTemperature()`	Ask the sensor for the temperature in °C
`isnan(temp)`	"Is Not A Number?" This is true if the reading failed
`return;`	Stop this turn of `loop()` early, then start again from the top

Try it: be a scientist

1. What temperature and humidity does your sensor show?

   _____

2. **Breathe** gently on the sensor for 5 seconds. What happens to the humidity? Why?

   _____

3. Hold the sensor between your fingers (not on the pins). How high does the temperature get?

   _____

Serial Monitor says: Could not read the sensor?

Check the S wire is on pin **4**. Wait a few seconds after uploading, because the sensor is slow to wake up.

4. Sketch 2: Weather station

Now let's show the readings on the screen, so your weather station works without a computer.

No computer needed

Once it's uploaded, your sketch stays in the ESP32. Unplug it from the computer and power it from a USB phone charger. Now it's a real weather station you can put anywhere!

Save as **L11_Station**:

Type this in → save it as **L11_Station** — 37 lines

```
// Lesson 11: Weather station
// DHT11 on pin 4, OLED screen on pins 21 (SDA) and 22 (SCL).

#include <DHT.h>
#include <Adafruit_SSD1306.h>

DHT dht(4, DHT11);
Adafruit_SSD1306 display(128, 64, &Wire, -1);

void setup() {
  dht.begin();
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
```

```
  display.setTextColor(SSD1306_WHITE);
}

void loop() {
  float temp = dht.readTemperature();
  float humidity = dht.readHumidity();

  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Weather station");
  if (isnan(temp) || isnan(humidity)) {
    display.println("Sensor error!");
  } else {
    display.setTextSize(2);
    display.setCursor(0, 18);
    display.print(temp, 1);
    display.print(" C");
    display.setCursor(0, 42);
    display.print(humidity, 0);
    display.print(" %");
  }
  display.display();
  delay(2000);
}
```

Code	What it means
Two `#include` lines	One sketch can use **two libraries** at once
`display.print(temp, 1);`	Print with **1** number after the decimal point: 23.4
`display.print(humidity, 0);`	Print with **no** decimal places: 45
`if (…) { … } else { … }`	If the reading failed, show an error. **Otherwise** show the numbers

5. Sketch 3: Comfort face

Let's make the weather station tell us **how the room feels**, with a smiley or a sad face.

Save as **L11_Comfort**:

Type this in → save it as **L11_Comfort** — 57 lines

```
// Lesson 11: Comfort face
// A smiley face when the room is comfortable, a sad face when not.

#include <DHT.h>
#include <Adafruit_SSD1306.h>

DHT dht(4, DHT11);
Adafruit_SSD1306 display(128, 64, &Wire, -1);
```

```
void drawFace(bool happy) {
  if (happy) {
    display.fillCircle(100, 38, 10, SSD1306_WHITE);  // smile
    display.fillCircle(100, 34, 10, SSD1306_BLACK);
  } else {
    display.fillCircle(100, 52, 10, SSD1306_WHITE);  // frown
    display.fillCircle(100, 56, 10, SSD1306_BLACK);
  }
  display.drawCircle(100, 36, 22, SSD1306_WHITE);
  display.fillCircle(92, 28, 3, SSD1306_WHITE);
  display.fillCircle(108, 28, 3, SSD1306_WHITE);
}

void setup() {
  dht.begin();
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.setTextColor(SSD1306_WHITE);
}

void loop() {
  float temp = dht.readTemperature();
  float humidity = dht.readHumidity();

  String message = "Just right!";
  if (temp > 26) {
    message = "Too hot!";
  } else if (temp < 18) {
    message = "Too cold!";
  } else if (humidity > 60) {
    message = "Too damp!";
  } else if (humidity < 30) {
    message = "Too dry!";
  }

  display.clearDisplay();
  display.setTextSize(2);
  display.setCursor(0, 4);
  display.print(temp, 0);
  display.println(" C");
  display.print(humidity, 0);
  display.println(" %");
  display.setTextSize(1);
  display.setCursor(0, 52);
  display.print(message);
  drawFace(message == "Just right!");
  display.display();
  delay(2000);
}
```

Code	What it means
`String message = "Just right!";`	A **String** variable holds text. We start by guessing "Just right!"
`if … else if … else if …`	Check one thing after another. Only the **first** true one happens
`message == "Just right!"`	`==` compares: is the message *exactly* "Just right!"? This is either true or false
`void drawFace(bool happy)`	A **bool** is just true or false. The function draws a smile for true and a frown for false

Big idea: the black circle trick

There's no "draw a smile" command! So we draw a white circle, then a **black** circle slightly higher on top of it. Only a thin crescent of white is left: a smile. For a frown, the black circle goes slightly **lower**.

Try it

1. Breathe on the sensor until the face turns sad. What message do you get?
2. Change the numbers so that 22 °C counts as "Too hot!". Can you make the face sad by holding the sensor?

6. Challenges

Challenge 1: Fahrenheit

Show the temperature in °F as well. *Hint:* `dht.readTemperature(true)` *gives Fahrenheit.*

Challenge 2: Record breaker

Remember the **lowest** and **highest** temperature since the ESP32 started, and show them on the screen. *Hint: start with* `float lowest = 100;`.

Challenge 3: Warning light

Add a red LED (with a 220 Ω resistor) on pin **26** that switches on when it's "Too hot!".

Bonus: Thermistor thermometer

A **thermistor** is a resistor that changes with temperature. Wire: **3V3 → 10 kΩ resistor → pin 34 → thermistor → GND**. Read it with `analogReadMilliVolts(34)` and watch the number change when you hold it. Ask your teacher for the formula to turn it into °C, and compare it with the DHT11!

7. Quiz

Check what you learned

1. What two things does the DHT11 measure?

   ---

2. What is the difference between an `int` and a `float`?

   ---

3. Why did the humidity go up when you breathed on the sensor?

Tidy up: unplug the USB cable and put the DHT11 back in the kit. You can leave the screen wired for later lessons.

LESSON 12

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Bat Ears

Measure distance with sound, then build a car parking sensor.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How the **ultrasonic sensor** uses echoes to measure distance
- How to time a pulse with `pulseIn()`
- Why a **voltage divider** keeps 5 volts away from the ESP32
- How to make a **function** that gives back a value with `return`

SESSION PLAN

0:00	How bats and dolphins "see" with sound
0:15	Build the sensor circuit with the voltage divider
0:35	Sketch 1: **L12_Distance**
0:55	Short break
1:00	Experiments: how far, how small, how soft?
1:15	Sketch 2: **L12_Parking** (add the buzzer)
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Ultrasonic sensor (HC-SR04, the one with two "eyes")
- ☐ 1 resistor, 1 kΩ (brown-black-red)
- ☐ 1 resistor, 2 kΩ (red-black-red)
- ☐ Passive buzzer (green board underneath)
- ☐ Jumper wires
- ☐ Extra: OLED screen from Lesson 10

1. Seeing with sound

Bats hunt in the dark by shouting very high squeaks and listening for the **echo**. The longer the echo takes to come back, the further away the moth is.

The ultrasonic sensor does exactly the same thing. One "eye" (**T**, the transmitter) is a tiny speaker. The other (**R**, the receiver) is a tiny microphone. The sound is **ultrasonic**: 40 000 vibrations per second, far too high for humans (or your dog) to hear.

> **Big idea: distance = speed × time**
>
> Sound travels about **343 metres per second**. That is **0.0343 cm every microsecond** (a microsecond is a millionth of a second). The sound goes *there and back*, so we divide by 2:
>
> **distance in cm = echo time × 0.0343 ÷ 2**

The sensor has 4 pins:

Pin	Job
VCC	Power: needs **5 V**
Trig	"Trigger": we send a short pulse here to say *shout now!*
Echo	The sensor holds this pin HIGH until it hears the echo
GND	Ground

2. Build the circuit

The sensor runs on 5 V, so its **Echo** pin sends back 5 V. That is too much for the ESP32, whose pins only like **3.3 V**. We use two resistors as a **voltage divider**: they share the 5 V between them so only about 3.3 V reaches the ESP32.

> **Don't skip the resistors!**
>
> Connecting Echo straight to the ESP32 can damage pin 16 over time. The divider takes 1 minute to build and keeps your board safe.

✓	From	To
☐	ESP32 **GND**	breadboard **–** rail
☐	ESP32 **5V** (may say **VIN**)	breadboard **+** rail
☐	Sensor **VCC**	**+** rail (5 V)
☐	Sensor **GND**	**–** rail
☐	Sensor **Trig**	ESP32 pin **17**
☐	Sensor **Echo**	an empty breadboard row (call it *row E*)
☐	**1 kΩ** resistor	from *row E* to another empty row (call it *row M*, the middle)
☐	**2 kΩ** resistor	from *row M* to the **–** rail
☐	*row M*	ESP32 pin **16**

> **Check the + rail**
>
> In earlier lessons the **+** rail may have been 3.3 V. Today it is **5 V**. Only the sensor's VCC should use it.

Point the sensor's eyes towards the edge of the table, with nothing in front of it.

3. Sketch 1: Measure the distance

File → New Sketch, type this in and save it as **L12_Distance**. Upload, then open the **Serial Monitor** at 115200 baud.

Type this in → save it as **L12_Distance** 37 lines

```
// Lesson 12: Measure distance with sound
// HC-SR04: TRIG on pin 17, ECHO on pin 16 (through the divider).

const int TRIG_PIN = 17;
const int ECHO_PIN = 16;

float readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);   // a 10 microsecond "ping"
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long echoTime = pulseIn(ECHO_PIN, HIGH, 30000);
  if (echoTime == 0) {
    return -1;                    // no echo came back
  }
  return echoTime * 0.0343 / 2;
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}

void loop() {
  float cm = readDistance();
  if (cm < 0) {
    Serial.println("Nothing in range");
  } else {
    Serial.print("Distance: ");
    Serial.print(cm);
    Serial.println(" cm");
  }
  delay(200);
}
```

Hold a book in front of the sensor and move it slowly closer and further away.

How it works

Code	What it means
`float readDistance() { … }`	A **function** we made up that *gives back* a `float` (a number with a decimal point)
`delayMicroseconds(10);`	Wait 10 *micro*seconds: 100 000 times shorter than `delay(1000)`!
`digitalWrite(TRIG_PIN, HIGH);`	A tiny pulse that tells the sensor: *shout now!*
`pulseIn(ECHO_PIN, HIGH, 30000)`	Stopwatch: how many microseconds does the Echo pin stay HIGH? Gives up after 30 000 µs
`long echoTime`	A `long` is like an `int`, but it can hold much bigger numbers
`return echoTime * 0.0343 / 2;`	Work out the distance and **send it back** to whoever called the function
`return -1;`	No echo means nothing is in range. We use –1 as a secret code for "*I don't know*"
`float cm = readDistance();`	Call the function and keep its answer in `cm`

Experiments

1. Use a ruler. Is the sensor accurate? How many cm is it off by?

   _____

2. What is the **closest** and **furthest** distance it can measure?

   _____

3. Try a pencil, your hand, a cushion and a wall. Which ones are hard to "see"? Why do you think soft things are tricky?

   _____

   _____

Only seeing 0 or *Nothing in range*?

Check that the sensor's VCC goes to **5V**, not 3V3. Check the divider: Echo → 1 kΩ → row M → 2 kΩ → GND, and row M → pin 16.

4. Sketch 2: Parking sensor

Cars have sensors like this in their bumpers. They beep slowly when a wall is far away, faster as it gets closer, and non-stop when you must STOP!

Add the **passive buzzer** (the one with the green board underneath):

✓	From	To
☐	Buzzer **+** (or **S**)	ESP32 pin **33**
☐	Buzzer **–**	**–** rail

Use **File** → **Save As…** to save a copy of your sketch as **L12_Parking**, then change it to match this:

Type this in → save it as **L12_Parking** 42 lines

```
// Lesson 12: Parking sensor
// Beeps faster as you get closer. Passive buzzer on pin 33.

const int TRIG_PIN = 17;
const int ECHO_PIN = 16;
const int BUZZER_PIN = 33;

float readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long echoTime = pulseIn(ECHO_PIN, HIGH, 30000);
  if (echoTime == 0) {
    return -1;
  }
  return echoTime * 0.0343 / 2;
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}

void loop() {
  float cm = readDistance();
  Serial.println(cm);

  if (cm < 0 || cm > 50) {
    noTone(BUZZER_PIN);           // far away: stay quiet
    delay(200);
  } else if (cm < 5) {
    tone(BUZZER_PIN, 1000);       // very close: non-stop beep
    delay(100);
  } else {
    tone(BUZZER_PIN, 1000, 60);   // short beep...
    delay(cm * 10);               // ...gap gets shorter when close
  }
}
```

Code	What it means
`cm < 0 || cm > 50`	`||` means **OR**: *nothing in range* OR *further than 50 cm*
`else if (cm < 5)`	Checked only if the first test was false
`tone(BUZZER_PIN, 1000, 60);`	Play a 1000 Hz note for 60 ms
`delay(cm * 10);`	Wait longer when the distance is bigger. 40 cm → 400 ms, 10 cm → 100 ms

Try it

1. Make the buzzer start beeping when you are 1 metre away.
2. Change the beep's **pitch** too, so it gets higher as you get closer.

5. Challenges

Challenge 1: Traffic light distance

Add your red, yellow and green LEDs (pins 25, 26, 27). Green when far, yellow when getting close, red when too close.

Challenge 2: Inches

Add a second function `readInches()` that uses `readDistance()` and divides by 2.54. Print both.

Challenge 3: Air instrument

Make a **theremin**: no beeping, just one note that changes pitch as your hand moves. *Hint:* `map()` *the distance 3–40 cm to a note of 1500–200 Hz.*

Bonus: Screen readout

Connect the OLED from Lesson 10 (SDA 21, SCL 22). Show the distance in big numbers, plus a bar that shrinks as things come closer.

6. Quiz

Check what you learned

1. Why do we divide the echo time by 2?

   _____

2. Why do we need the 1 kΩ and 2 kΩ resistors?

   _____

   _____

3. An echo takes 1000 µs. How far away is the object? (Use the formula!)

   _____

Tidy up: unplug the USB cable. Keep the sensor's divider resistors together so you can find them again.

LESSON 13

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Intruder Alert!

Build a motion-sensing burglar alarm that you can arm and disarm.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **PIR sensor** feels the heat of moving bodies
- How to spot the moment something **changes** (an *edge*)
- How to use a `bool` variable to remember if the alarm is **armed**
- How to tune a sensor with its adjustment knobs

SESSION PLAN

0:00	How do motion sensors work?
0:10	Wire up the PIR sensor and LED
0:25	Sketch 1: **L13_Motion** (+ 1 minute warm-up)
0:40	Knob experiments: range and timer
0:55	Short break
1:00	Add LEDs, button and buzzer
1:15	Sketch 2: **L13_Alarm**
1:35	Challenges: test your alarm on a friend!
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ PIR motion sensor (HC-SR501, white dome)
- ☐ 1 red LED and 1 green LED
- ☐ 2 resistors, 220 Ω (red-red-brown)
- ☐ 1 push button
- ☐ Passive buzzer
- ☐ Jumper wires (3 female-to-male for the PIR)

1. How does it see you?

Everything warm gives off invisible **infrared** light. People, cats and dogs glow brightly in infrared; walls and tables are dim. The **PIR** (Passive InfraRed) sensor has a heat-sensitive chip under its white dome.

The dome is a clever lens made of lots of little windows. When you walk past, your warm body moves from one window to the next, and the sensor notices the *change*. That is why it spots **moving** people but not a person who sits perfectly still.

> **Big idea: the sensor does the hard work**
>
> The PIR sensor has its own little chip. It sends the ESP32 one simple signal: pin **HIGH** = "*I saw movement!*", pin **LOW** = "*all quiet*". Our code just has to read one pin.

2. Build the motion detector

Take the PIR sensor and carefully lift off the white dome to have a look (then put it back!). Find the 3 pins and their labels: **VCC**, **OUT**, **GND**.

Turn the sensor over. You'll find **two orange knobs** (use a small screwdriver or your fingernail):

- One is **SENSITIVITY**: how far away it can see (about 3 to 7 metres).
- One is **TIME**: how long OUT stays HIGH after it sees you (about 3 seconds to 5 minutes!).

Some boards have a little **jumper** too. If yours does, put it on **H** ("repeat"), so the timer keeps restarting while you keep moving.

✓	From	To
☐	ESP32 **GND**	breadboard **–** rail
☐	ESP32 **5V** (may say **VIN**)	breadboard **+** rail
☐	PIR **VCC**	**+** rail (5 V)
☐	PIR **GND**	**–** rail
☐	PIR **OUT**	ESP32 pin **27**
☐	ESP32 pin **26**	220 Ω resistor → **long leg** of the **green** LED
☐	short leg of the green LED	**–** rail

> **The PIR is happy with 5 V**
>
> It needs 5 V to power itself, but its **OUT** pin only ever sends 3.3 V. So it is safe to connect straight to the ESP32, with no divider needed.

3. Sketch 1: Motion detector

> **Wait one minute!**
>
> When it first gets power, the PIR sensor needs about **1 minute** to learn what the room looks like. During this time it may trigger randomly. Keep still and be patient!

File → New Sketch, type this in, save it as **L13_Motion**, upload, and open the **Serial Monitor**.

Type this in → save it as **L13_Motion** — 32 lines

```
// Lesson 13: Motion detector
// PIR sensor on pin 27, LED on pin 26.

const int PIR_PIN = 27;
const int LED_PIN = 26;

int count = 0;
bool wasMoving = false;

void setup() {
  Serial.begin(115200);
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("Warming up... keep still for 1 minute!");
}
```

```
void loop() {
  bool moving = digitalRead(PIR_PIN) == HIGH;
  digitalWrite(LED_PIN, moving);

  if (moving && !wasMoving) {       // it just started
    count = count + 1;
    Serial.print("Motion! Count: ");
    Serial.println(count);
  }
  if (!moving && wasMoving) {       // it just stopped
    Serial.println("All quiet.");
  }

  wasMoving = moving;
  delay(50);
}
```

How it works

Code	What it means
`bool wasMoving = false;`	A `bool` holds only **true** or **false**. This one remembers what we saw *last time*
`digitalRead(PIR_PIN) == HIGH`	Is the sensor saying "movement"? The answer (true/false) goes in `moving`
`digitalWrite(LED_PIN, moving);`	`true` works like HIGH and `false` like LOW, so the LED copies the sensor
`moving && !wasMoving`	`&&` means **AND**, `!` means **NOT**. *Moving now* AND *not moving before* = it **just started**
`wasMoving = moving;`	Remember this reading for the next time round the loop

> **Big idea: catching the *edge***
>
> The sensor stays HIGH for a few seconds. If we added 1 to `count` *every time* we saw HIGH, it would count hundreds per wave! Instead, we only count at the **edge**: the moment it changes from LOW to HIGH.

4. Tune your sensor

Remember the two orange knobs underneath the sensor? Now that your sketch is running, find out which knob is which.

Detective work

1. Turn **one** knob fully anticlockwise. Wave, then stand still. How long does the LED stay on? Which knob did you turn?

   ---

2. Set the TIME knob to its **shortest** setting (you'll need it short for the alarm). How far across the room can the sensor see you?

   ---

3. After the LED goes off, the sensor ignores you for about 3 seconds. Can you find this "blind" moment?

   ---

5. Sketch 2: Burglar alarm

Now let's make a real alarm. The **button** arms and disarms it. **Green** = disarmed (safe), **red** = armed. If someone moves while it's armed, the siren wails!

Add these parts:

✓	From	To
☐	ESP32 pin **25**	220 Ω resistor → **long leg** of the **red** LED
☐	short leg of the red LED	**–** rail
☐	ESP32 pin **13**	one leg of the **button**
☐	the button's **opposite** leg	**–** rail
☐	Buzzer **+**	ESP32 pin **33**
☐	Buzzer **–**	**–** rail

File → **New Sketch**, type this in and save it as **L13_Alarm**.

Type this in → save it as **L13_Alarm** — 65 lines

```
// Lesson 13: Burglar alarm
// Button on 13 arms/disarms. Red LED 25, green LED 26, buzzer 33.

const int PIR_PIN = 27;
const int BUTTON_PIN = 13;
const int RED_PIN = 25;
const int GREEN_PIN = 26;
const int BUZZER_PIN = 33;

bool armed = false;
bool wasMoving = false;
int alarms = 0;

void showState() {
  digitalWrite(RED_PIN, armed);
  digitalWrite(GREEN_PIN, !armed);
```

```
  if (armed) {
    Serial.println("ARMED");
  } else {
    Serial.println("Disarmed");
  }
}

void siren() {
  for (int i = 0; i < 6; i++) {
    digitalWrite(RED_PIN, HIGH);
    tone(BUZZER_PIN, 1200);
    delay(250);
    digitalWrite(RED_PIN, LOW);
    tone(BUZZER_PIN, 800);
    delay(250);
  }
  noTone(BUZZER_PIN);
  showState();
}

void setup() {
  Serial.begin(115200);
  pinMode(PIR_PIN, INPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(RED_PIN, OUTPUT);
  pinMode(GREEN_PIN, OUTPUT);
  showState();
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {
    armed = !armed;
    showState();
    tone(BUZZER_PIN, 2000, 100);
    while (digitalRead(BUTTON_PIN) == LOW) {
      delay(10);                    // wait for the finger to lift
    }
  }

  bool moving = digitalRead(PIR_PIN) == HIGH;
  if (armed && moving && !wasMoving) {
    alarms = alarms + 1;
    Serial.print("INTRUDER! Alarm number ");
    Serial.println(alarms);
    siren();
  }
  wasMoving = moving;
  delay(20);
}
```

Code	What it means
`armed = !armed;`	Flip it: true becomes false, false becomes true
`digitalWrite(GREEN_PIN, !armed);`	Green is on when the alarm is **not** armed
`void siren() { … }`	Our own function: flash red and switch between two notes, 6 times
`while (digitalRead(BUTTON_PIN) == LOW)`	Keep waiting **while** the button is still held down
`armed && moving && !wasMoving`	Armed, AND moving now, AND not moving before

The alarm goes off as soon as I arm it!

You're too close and moving! Arm it, then **freeze** until the PIR's timer runs out. Or point the sensor away from you. (Challenge 2 fixes this properly.)

6. Challenges

Challenge 1: Silent alarm

Make a "spy mode" where the alarm only flashes the red LED and prints a message, with no sound, so the intruder doesn't know they've been spotted.

Challenge 2: Exit delay

Real alarms give you time to leave. After arming, beep once a second for 10 seconds before the alarm becomes active. *Hint: a* `for` *loop with* `tone()` *and* `delay(1000)`*.*

Challenge 3: Alarm diary

When the alarm goes off, print how long after arming it happened, like `Alarm 2 at 01:35`. *Hint: save* `millis()` *when you arm it, then subtract.*

Bonus: Test it on a friend

Hide your alarm guarding a "treasure". Can a friend sneak up and grab it without setting it off? What tricks work against a PIR sensor?

7. Quiz

Check what you learned

1. Why does the PIR see a walking person but not a statue of a person?

2. What does `!` mean in code? What is `!true`?

3. Why do we count motion only when `moving && !wasMoving`?

Tidy up: unplug the USB cable. Put the PIR sensor's dome back on if you took it off.

LESSON 14

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Robot Arm

Make a servo motor turn to any angle and steer it with a joystick.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **servo** turns to an exact angle
- How to use a **library** to control a servo
- How a **joystick** is really two knobs and a button
- How to keep a number between limits with `constrain()`

SESSION PLAN

0:00	What's inside a servo?
0:10	Install the **ESP32Servo** library
0:15	Wire the servo, Sketch 1: **L14_Sweep**
0:35	Wire the joystick, Sketch 2: **L14_Joystick**
0:55	Short break
1:00	Sketch 3: **L14_Pointer** (steer the servo)
1:20	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Servo motor SG90 (small blue motor) + a horn (the plastic arm)
- ☐ Joystick module
- ☐ 5 female-to-male jumper wires (for the joystick)
- ☐ Jumper wires

1. Meet the servo

A normal motor just spins round and round. A **servo** is smarter: you tell it an **angle** between 0° and 180°, and it turns to that angle and *holds it there*. Servos move robot arms, steer remote-control cars, and flap the wings on animatronic dinosaurs.

Inside the blue box are a small motor, a stack of gears (to make it strong), a knob that measures where the arm is, and a tiny circuit that keeps correcting the motor until the arm is at the right angle.

> **Big idea: angles in pulses**
>
> The ESP32 tells the servo which angle it wants by sending a pulse 50 times a second. A **short** pulse (0.5 ms) means 0°, a **long** pulse (2.4 ms) means 180°. A **library** does all the timing for us. We just write `myServo.write(90);`

2. Install the library

A **library** is code someone else wrote that we can borrow.

- ☐ **Tools → Manage Libraries…**
- ☐ Search for **ESP32Servo** and click **Install** on *ESP32Servo by Kevin Harrington, John K. Bennett*.

3. Wire the servo

The servo has 3 wires bundled in one plug. Push male jumper wires into the plug's holes.

✓	From	To
☐	ESP32 **GND**	breadboard **–** rail
☐	ESP32 **5V** (may say **VIN**)	breadboard **+** rail
☐	Servo **brown** wire (GND)	**–** rail
☐	Servo **red** wire (power)	**+** rail (5 V)
☐	Servo **orange** wire (signal)	ESP32 pin **18**

Press one of the plastic **horns** onto the servo's shaft so you can see it move.

> **Don't force it!**
>
> Never twist the servo arm by hand when it is powered. You can strip its tiny plastic gears.

4. Sketch 1: Sweep

File → New Sketch, type this in, save as **L14_Sweep**, and upload.

Type this in → save it as **L14_Sweep** — 22 lines

```
// Lesson 14: Servo sweep
// Servo signal (orange wire) on pin 18.

#include <ESP32Servo.h>

const int SERVO_PIN = 18;
Servo myServo;

void setup() {
  myServo.attach(SERVO_PIN, 500, 2400);
}

void loop() {
  for (int angle = 0; angle <= 180; angle++) {
    myServo.write(angle);
    delay(15);
  }
  for (int angle = 180; angle >= 0; angle--) {
    myServo.write(angle);
    delay(15);
  }
}
```

Code	What it means
`#include <ESP32Servo.h>`	Borrow the servo library's code
`Servo myServo;`	Make a servo **object** called `myServo` (like a variable, but it can *do* things)
`myServo.attach(SERVO_PIN, 500, 2400);`	The servo is on pin 18. Pulses go from 500 to 2400 microseconds
`myServo.write(angle);`	Turn to this angle (0–180)
`angle--`	Short for `angle = angle - 1` (and `angle++` adds 1)

Try it

1. Change `delay(15)` to `delay(5)` and to `delay(40)`. What changes?
2. Make it sweep only between 45° and 135°, like a windscreen wiper.

ESP32 restarts or the servo jitters?

The servo can briefly need more power than the USB port gives. Try a different USB port or a powered USB hub. Make sure the servo's brown wire is in **GND**.

5. Wire the joystick

A **joystick** is really **two knobs** (like the potentiometer from Lesson 4) plus a **button** you press by clicking the stick down. One knob measures left–right (**X**), the other up–down (**Y**).

✓	Joystick pin	To
☐	**GND**	**–** rail
☐	**+5V**	ESP32 **3V3** pin (yes, **3V3**, not 5V!)
☐	**VRx**	ESP32 pin **34**
☐	**VRy**	ESP32 pin **35**
☐	**SW**	ESP32 pin **14**

Use 3V3 for the joystick

Even though the joystick's pin says **+5V**, connect it to **3V3**. The ESP32 can only measure voltages up to 3.3 V, and 5 V would damage pins 34 and 35.

6. Sketch 2: Read the joystick

Type this in as **L14_Joystick**, upload, and open the **Serial Monitor**.

Type this in → save it as **L14_Joystick** 27 lines

```
// Lesson 14: Read the joystick
// VRx on 34, VRy on 35, SW (button) on 14.

const int X_PIN = 34;
const int Y_PIN = 35;
const int SW_PIN = 14;

void setup() {
  Serial.begin(115200);
  pinMode(SW_PIN, INPUT_PULLUP);
}

void loop() {
  int x = analogRead(X_PIN);
  int y = analogRead(Y_PIN);
  bool pressed = digitalRead(SW_PIN) == LOW;

  Serial.print("X: ");
  Serial.print(x);
  Serial.print("  Y: ");
  Serial.print(y);
  if (pressed) {
    Serial.print("  CLICK!");
  }
  Serial.println();
  delay(100);
}
```

Map your joystick

Write down the numbers you see:

Stick position	X number	Y number
Middle (let go)		
Full left		
Full right		
Full up		
Full down		

The numbers go from **0** to **4095**. Do you remember from Lesson 4 why? (The ESP32 measures in 4096 tiny steps.) The middle is *about* 1900, but it's never perfectly still. Every joystick wobbles a bit.

7. Sketch 3: Steer the servo

Now let's join them! Pushing the stick one way turns the servo a little bit every 20 ms; pushing it the other way turns it back. Clicking the stick sends the servo back to the middle.

File → **New Sketch**, type this in, and save as **L14_Pointer**.

Type this in → save it as **L14_Pointer** — 36 lines

```
// Lesson 14: Joystick-controlled servo
// Push the stick left/right to turn. Click to go back to the middle.

#include <ESP32Servo.h>

const int SERVO_PIN = 18;
const int X_PIN = 34;
const int SW_PIN = 14;

Servo myServo;
int angle = 90;

void setup() {
  Serial.begin(115200);
  pinMode(SW_PIN, INPUT_PULLUP);
  myServo.attach(SERVO_PIN, 500, 2400);
  myServo.write(angle);
}

void loop() {
  int x = analogRead(X_PIN);

  if (x < 1000) {
    angle = angle + 2;              // stick pushed one way
  } else if (x > 3000) {
    angle = angle - 2;              // stick pushed the other way
  }
  if (digitalRead(SW_PIN) == LOW) {
    angle = 90;
  }

  angle = constrain(angle, 0, 180);
  myServo.write(angle);
  Serial.println(angle);
  delay(20);
}
```

Code	What it means
`int angle = 90;`	Remember where the servo is. Start in the middle
`if (x < 1000)`	The stick is pushed far to one side. Anything between 1000 and 3000 counts as "middle", so small wobbles are ignored
`angle = angle + 2;`	Turn 2° more. Do this 50 times a second and it turns smoothly
`constrain(angle, 0, 180)`	Keep `angle` between 0 and 180. If it's 184, it becomes 180

Turns the wrong way?

Swap `+ 2` and `- 2`. Or turn your joystick around!

8. Challenges

Challenge 1: Speed control

The further you push the stick, the faster the servo turns. *Hint: use* `map()` *to turn the X value into a step size between –5 and +5.*

Challenge 2: Direct drive

Instead of turning step by step, make the servo angle *copy* the stick: `angle = map(x, 0, 4095, 0, 180);` What feels better for a robot arm?

Challenge 3: Robot memory

Click to save the current angle in an **array**. After 5 clicks, make the servo play back all 5 positions in order, 3 times, like a robot repeating a dance. Then use card and tape to turn your servo into a waving robot arm!

9. Quiz

Check what you learned

1. What is the difference between a normal motor and a servo?

2. What does `constrain(200, 0, 180)` give back?

3. Why must the joystick be powered from **3V3** and not 5V?

Tidy up: unplug the USB cable. Take the horn off the servo and keep it in the servo's bag.

LESSON 15

Motor Mania

Spin a fan at any speed in both directions, then make a stepper motor move one tiny step at a time.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- Why motors need their **own power supply**
- How a **motor driver chip** (L293D) lets the ESP32 control a motor
- How to change **speed** with PWM and **direction** with two pins
- How a **stepper motor** moves in exact steps

SESSION PLAN

0:00	Why can't the ESP32 power a motor?
0:10	Set up the power supply module
0:20	Wire the L293D and the fan motor
0:40	Sketch 1: **L15_Fan**
0:55	Short break
1:00	Wire the stepper, Sketch 2: **L15_Stepper**
1:25	Stepper experiments
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Power supply module + 9V battery with clip (or a 2nd USB cable)
- ☐ L293D chip (16 legs)
- ☐ DC motor (3–6 V) + fan blade
- ☐ Stepper motor (28BYJ-48) + ULN2003 driver board
- ☐ Jumper wires (6 female-to-male for the stepper board)
- ☐ Extra: potentiometer (for a challenge)

1. Motors are hungry!

An ESP32 pin can give out a tiny amount of electricity, enough for an LED. A motor needs **10 to 20 times more**! If you connected a motor straight to a pin, it would hardly turn, and the ESP32 could be damaged.

So we split the job in two:

- The **ESP32** is the **brain**. It sends small "go/stop/faster" signals.
- A **driver chip** is the **muscle**. It takes those signals and switches a big current from a separate **power supply** through the motor.

Safety rules for today

- Motor power comes **only** from the power supply module, **never** from the ESP32's 3V3 or 5V pins.
- Switch the power module **off** while you change wires.
- Keep fingers and hair away from the spinning fan. Hold or tape the motor down.
- If anything gets hot or smells funny: switch off and unplug **everything**, then call your teacher.

2. Set up the power supply module

The power supply module plugs straight into the end of a breadboard. It turns a 9 V battery (or a USB cable) into a steady **5 V** on the breadboard rails.

✓	Step
☐	**Remove** any wires from the ESP32's **5V** or **3V3** pins to the breadboard rails. (Two power supplies must never be joined!)
☐	Push the module onto the end of one breadboard, so its pins sit in the **+** and **–** rails (match **+** to **+**!)
☐	Set **both** yellow jumpers on the module to **5V**
☐	Connect a wire from an ESP32 **GND** pin to the module's **–** rail
☐	Clip the 9 V battery to its plug and plug it into the module (or use a USB cable). Leave the switch **off** for now

> **Big idea: share the ground**
>
> The ESP32 and the power module have separate **+**, but they must share **GND**. GND is the "zero" that both measure their voltages from. Without a shared GND, the driver chip can't understand the ESP32's signals.

3. Wire the fan motor

The **L293D** chip has 16 legs. Find the little **notch** (a half-circle) at one end. With the notch at the top, leg 1 is top-left. Count **down** the left side 1 to 8, then **up** the right side 9 to 16.

Push the chip into a free part of a breadboard, **across the middle gap**, so each leg has its own row.

✓	From	To
☐	L293D leg **1** (Enable: speed)	ESP32 pin **25**
☐	L293D leg **2** (Input 1)	ESP32 pin **26**
☐	L293D leg **3** (Output 1)	one **motor** wire
☐	L293D legs **4** and **5** (GND)	**–** rail
☐	L293D leg **6** (Output 2)	the other **motor** wire
☐	L293D leg **7** (Input 2)	ESP32 pin **27**
☐	L293D leg **8** (motor power)	**+** rail (5 V from module)
☐	L293D leg **16** (chip power)	**+** rail (5 V from module)

Push the fan blade onto the motor's shaft. Check every wire twice, then switch the module **on**.

4. Sketch 1: Speed and direction

File → **New Sketch**, type this in, save as **L15_Fan**, upload, and open the **Serial Monitor**.

Type this in → save it as **L15_Fan** 41 lines

```
// Lesson 15: DC motor fan with the L293D chip
// EN (speed) on 25, IN1 on 26, IN2 on 27.

const int EN_PIN = 25;
const int IN1_PIN = 26;
const int IN2_PIN = 27;

void motor(int speed, bool forward) {
  digitalWrite(IN1_PIN, forward);
  digitalWrite(IN2_PIN, !forward);
  analogWrite(EN_PIN, speed);       // 0 = stop, 255 = full speed
}

void setup() {
  Serial.begin(115200);
  pinMode(EN_PIN, OUTPUT);
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);
}

void loop() {
  Serial.println("Forward, slow");
  motor(150, true);
  delay(3000);

  Serial.println("Forward, fast");
  motor(255, true);
  delay(3000);

  Serial.println("Stop");
  motor(0, true);
  delay(2000);

  Serial.println("Backward");
  motor(200, false);
  delay(3000);

  Serial.println("Stop");
  motor(0, true);
  delay(2000);
}
```

How it works

Code	What it means
`IN1 HIGH`, `IN2 LOW`	Current flows through the motor one way: it spins **forwards**
`IN1 LOW`, `IN2 HIGH`	Current flows the other way: it spins **backwards**
`digitalWrite(IN2_PIN, !forward);`	IN2 is always the *opposite* of IN1
`analogWrite(EN_PIN, speed);`	**PWM** from Lesson 5: switch the motor on and off very fast. More "on" time = faster
`void motor(int speed, bool forward)`	Our own function with **two inputs**, so `motor(150, true)` does all the work

Try it

1. What is the **lowest** speed that still makes your fan start from standing still?

   _____

2. Does the fan blow air at you when it goes forwards, or backwards? How could you swap that **without** changing the code?

   _____

Motor doesn't spin?

Is the module switched on, with jumpers on 5V? Is the battery fresh? Did you connect ESP32 GND to the **–** rail? Try `motor(255, true)`. Some motors won't start at slow speeds.

5. The stepper motor

A **stepper motor** doesn't spin freely. It moves in tiny, exact **steps**. Inside are 4 electromagnets in a circle. Switch them on in the right order and the middle is pulled round, step by step. Our stepper needs **2048 steps** for one full turn. That's how printers and 3D printers move so precisely!

The **ULN2003** board is the stepper's driver (its "muscle"). Its 4 little LEDs show you which magnets are on.

✓	From	To
☐	Switch the power module **off** first!	
☐	Stepper's **white plug**	ULN2003 white socket (it only fits one way)
☐	ULN2003 **IN1**	ESP32 pin **16**
☐	ULN2003 **IN2**	ESP32 pin **17**
☐	ULN2003 **IN3**	ESP32 pin **18**
☐	ULN2003 **IN4**	ESP32 pin **19**
☐	ULN2003 **–**	**–** rail
☐	ULN2003 **+**	**+** rail (5 V from module)

Stick a small piece of tape on the shaft like a flag so you can see it turn. Switch the module **on**.

6. Sketch 2: Step by step

File → **New Sketch**, type this in, and save as **L15_Stepper**.

Type this in → save it as **L15_Stepper** — 50 lines

```
// Lesson 15: Stepper motor
// ULN2003 board IN1–IN4 on pins 16, 17, 18, 19.

const int PINS[4] = {16, 17, 18, 19};
const int PATTERN[4][4] = {
  {1, 1, 0, 0},
  {0, 1, 1, 0},
  {0, 0, 1, 1},
  {1, 0, 0, 1}
};
const int STEPS_PER_TURN = 2048;

int stepNumber = 0;

void step(int direction) {
  stepNumber = (stepNumber + direction + 4) % 4;
  for (int i = 0; i < 4; i++) {
    digitalWrite(PINS[i], PATTERN[stepNumber][i]);
  }
  delay(3);
}

void allOff() {
  for (int i = 0; i < 4; i++) {
    digitalWrite(PINS[i], LOW);
  }
}

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 4; i++) {
    pinMode(PINS[i], OUTPUT);
  }
}

void loop() {
  Serial.println("One full turn forwards");
  for (int i = 0; i < STEPS_PER_TURN; i++) {
    step(1);
  }
  allOff();
  delay(1000);
```

```
  Serial.println("Quarter turn backwards");
  for (int i = 0; i < STEPS_PER_TURN / 4; i++) {
    step(-1);
  }
  allOff();
  delay(1000);
}
```

Code	What it means
`const int PATTERN[4][4]`	A **2D array**: a table with 4 rows and 4 columns. Each row says which of the 4 magnets is on
`PATTERN[stepNumber][i]`	Row `stepNumber`, column `i`
`(stepNumber + direction + 4) % 4`	`%` gives the **remainder** after dividing. It makes 0,1,2,3 go round in a circle: after 3 comes 0 again
`step(-1)`	Go through the pattern **backwards**, so the motor turns the other way
`delay(3);`	Wait 3 ms between steps. Too short and the motor can't keep up
`allOff()`	Switch all magnets off so the motor doesn't get warm while it waits

Experiments

1. Watch the 4 LEDs on the ULN2003 board. What pattern do you see? (Try `delay(300)` in `step()` to slow it right down.)

   _____

2. What is the smallest delay that still works? What happens if you go lower?

   _____

3. How many steps make **half** a turn? How many steps for 90°?

   _____

7. Challenges

Challenge 1: Fan control knob

Add the potentiometer (middle leg to pin **34**, outer legs to ESP32 **3V3** and **GND**). Knob in the middle = stop, turn right = forwards, turn left = backwards. Turn further = faster.

Challenge 2: Turn to an angle

Write a function `turnDegrees(int degrees)` that turns the stepper by any angle. Positive = forwards, negative = backwards. *Hint: steps = degrees × 2048 ÷ 360.*

Challenge 3: Stepper clock

Make a paper clock face and a paper seconds hand. Make the stepper go round **exactly once per minute**, ticking every second.

8. Quiz

Check what you learned

1. Why don't we connect a motor straight to an ESP32 pin?

   ---

2. Which two things do we change to make the fan spin the other way?

   ---

3. What is `10 % 4`?

Tidy up: switch the power module **off** and unplug the battery **first**, then unplug the USB cable. Unclip the battery so it doesn't run flat.

LESSON 16

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Secret Code Lock

Build a keypad door lock with a servo bolt, beeps and warning lights.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **keypad** finds which key you pressed using rows and columns
- How to use the **Keypad** library
- How to build up text in a `String` and compare it
- How to put everything together into a real gadget

SESSION PLAN

0:00	How does a keypad work?
0:10	Install the **Keypad** library and wire the keypad
0:30	Sketch 1: **L16_Keys**
0:45	Add the servo, LEDs and buzzer
0:55	Short break
1:00	Sketch 2: **L16_Lock**
1:25	Challenges
1:50	Crack your neighbour's code!
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ 4×4 membrane keypad
- ☐ Servo motor SG90 + horn
- ☐ Passive buzzer
- ☐ 1 red LED and 1 green LED
- ☐ 2 resistors, 220 Ω (red-red-brown)
- ☐ Jumper wires (8 for the keypad)

1. Sixteen keys, eight wires

The keypad has 16 keys but only **8 wires**. How can that work?

Under the keys are 4 **row** wires going across and 4 **column** wires going down. When you press a key, it connects one row to one column, like where two roads cross.

The ESP32 checks super fast: "*Row 1, are you touching any column? Row 2? Row 3?…*" When it finds a connection at row 2 and column 3, it knows you pressed **6**. This is called **scanning**, and it happens hundreds of times a second.

> **Big idea: rows × columns**
>
> 4 rows × 4 columns = 16 keys, using only 4 + 4 = 8 wires. Computer keyboards use the same trick to read 100 keys!

2. Install the library and wire the keypad

- ☐ **Tools** → **Manage Libraries…**, search **Keypad**, and install *Keypad by Mark Stanley, Alexander Brevig*.
- ☐ You'll need **ESP32Servo** too. You installed it in Lesson 14.

Hold the keypad face up with the ribbon at the bottom. The 8 holes in its plug are, **from left to right**: rows 1–4, then columns 1–4. Push a male jumper wire into each hole.

✓	Keypad hole (left → right)	ESP32 pin
☐	1 (row 1)	**13**
☐	2 (row 2)	**14**
☐	3 (row 3)	**27**
☐	4 (row 4)	**26**
☐	5 (column 1)	**25**
☐	6 (column 2)	**4**
☐	7 (column 3)	**16**
☐	8 (column 4)	**17**

Also connect the buzzer: **+** to pin **33**, **–** to the **–** rail, and ESP32 **GND** to the **–** rail.

> **Use wires of different colours**
>
> It is much easier to check 8 wires if they are in a rainbow order. Then you can see at a glance if two are swapped.

3. Sketch 1: Which key?

File → **New Sketch**, type this in and save as **L16_Keys**. Upload, open the **Serial Monitor**, and press every key.

Type this in → save it as **L16_Keys** — 33 lines

```
// Lesson 16: Read the keypad
// Rows on 13, 14, 27, 26. Columns on 25, 4, 16, 17. Buzzer on 33.

#include <Keypad.h>

const byte ROWS = 4;
const byte COLS = 4;
char keys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte rowPins[ROWS] = {13, 14, 27, 26};
byte colPins[COLS] = {25, 4, 16, 17};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

const int BUZZER_PIN = 33;
```

```
void setup() {
  Serial.begin(115200);
  Serial.println("Press some keys!");
}

void loop() {
  char key = keypad.getKey();
  if (key) {
    Serial.print("You pressed: ");
    Serial.println(key);
    tone(BUZZER_PIN, 1500, 50);
  }
}
```

Code	What it means
`#include <Keypad.h>`	Borrow the Keypad library
`char keys[ROWS][COLS]`	A `char` is **one** letter or symbol, in single quotes: `'7'`. This 2D array is a map of the keypad
`byte rowPins[ROWS]`	A `byte` is a small whole number (0–255), perfect for pin numbers
`Keypad keypad = Keypad(…)`	Make a keypad object that knows the map and the pins
`char key = keypad.getKey();`	Which key was just pressed? Gives back `0` (nothing) if none
`if (key)`	True if a key was pressed (anything except 0)

Wrong keys showing?

If pressing **1** shows **D** or *, your 8 wires are in the opposite order. Either turn the plug around, or reverse the numbers in `rowPins` and `colPins`. If one whole row is dead, check that row's wire.

4. Add the lock parts

✓	From	To
☐	ESP32 **5V** (may say **VIN**)	breadboard **+** rail
☐	Servo **brown**	**–** rail
☐	Servo **red**	**+** rail (5 V)
☐	Servo **orange**	ESP32 pin **18**
☐	ESP32 pin **19**	220 Ω resistor → **long leg** of **red** LED
☐	ESP32 pin **23**	220 Ω resistor → **long leg** of **green** LED
☐	short legs of both LEDs	**–** rail

The servo is the door's **bolt**: at 0° it is locked, at 90° it is open.

5. Sketch 2: The lock

File → New Sketch, type this in and save as **L16_Lock**. The secret code is **1234**. Type it, then press **#**.

Type this in → save it as **L16_Lock** — 70 lines

```
// Lesson 16: Secret code lock
// Type the code then #. Press * to start again.

#include <Keypad.h>
#include <ESP32Servo.h>

char keys[4][4] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte rowPins[4] = {13, 14, 27, 26};
byte colPins[4] = {25, 4, 16, 17};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, 4, 4);
const int BUZZER_PIN = 33;
const int RED_PIN = 19;
const int GREEN_PIN = 23;
const int SERVO_PIN = 18;

Servo lock;
String secret = "1234";
String typed = "";

void openLock() {
  Serial.println("Correct! Door open.");
  digitalWrite(GREEN_PIN, HIGH);
  lock.write(90);
  delay(5000);
  lock.write(0);
  digitalWrite(GREEN_PIN, LOW);
}

void wrongCode() {
  Serial.println("WRONG CODE!");
  digitalWrite(RED_PIN, HIGH);
  tone(BUZZER_PIN, 200, 800);
  delay(1000);
  digitalWrite(RED_PIN, LOW);
}

void setup() {
  Serial.begin(115200);
  pinMode(RED_PIN, OUTPUT);
```

```
  pinMode(GREEN_PIN, OUTPUT);
  lock.attach(SERVO_PIN, 500, 2400);
  lock.write(0);
}

void loop() {
  char key = keypad.getKey();
  if (!key) {
    return;                          // no key pressed: try again
  }
  tone(BUZZER_PIN, 1500, 50);
  if (key == '*') {
    typed = "";
    Serial.println("Cleared");
  } else if (key == '#') {
    if (typed == secret) {
      openLock();
    } else {
      wrongCode();
    }
    typed = "";
  } else {
    typed = typed + key;
    Serial.println(typed);
  }
}
```

How it works

Code	What it means
`String secret = "1234";`	A `String` holds **text**: lots of characters in double quotes
`String typed = "";`	Starts **empty**. It collects the keys you press
`typed = typed + key;`	Stick the new key on the end: `"12"` + `'3'` → `"123"`
`typed == secret`	Are the two texts exactly the same?
`return;`	Leave `loop()` early. It starts again straight away
`key == '*'`	Single quotes for one character, double quotes for a String

Try it

1. Change the secret code to your own. Can it have letters in it, like `"A7B9"`?
2. Make the door stay open for 10 seconds instead of 5.

6. Challenges

Challenge 1: Secret stars

Anyone watching the Serial Monitor can see your code! Print a `*` for each key instead of the code itself.

Challenge 2: Three strikes

Count wrong tries. After 3 wrong codes in a row, block the keypad for 30 seconds with the red LED flashing. A right code resets the count to 0.

Challenge 3: New code

While the door is open, pressing **A** lets you type a new secret code followed by **#**. *Hint: in* `openLock()`*, wait for keys in a loop instead of* `delay(5000)`*.*

Bonus: Code cracker

Swap computers with a friend (don't peek at their code!). The code is 3 digits. How many tries does it take to crack it? How many 3-digit codes are possible?

7. Quiz

Check what you learned

1. How many wires would a keypad with 5 rows and 6 columns need? How many keys would it have?

   ---

2. What is the difference between `'7'` and `"7"`?

   ---

3. What is in `typed` after you press **4**, **2**, *****, **9**?

Tidy up: unplug the USB cable. Gently pull the jumper wires out of the keypad plug (don't pull the ribbon!).

LESSON 17

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Secret Agent Cards

Use RFID to read a card without touching it, and build an access-control door.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How **RFID** works (radio + a tiny chip with no battery)
- How to read a card's unique ID, called its **UID**
- How to compare **Strings** and search an **array** of allowed cards
- How to build an access system with lights, sound and (bonus) a servo lock

SESSION PLAN

0:00	How does a bus pass work? RFID explained
0:15	Wire up the RC522 reader
0:35	Sketch 1: **L17_ReadCard** — find your card's UID
0:55	Short break
1:00	Add the LEDs and buzzer
1:10	Sketch 2: **L17_Access** — allowed or denied?
1:35	Challenges (servo door!)
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, 2 breadboards
- ☐ RC522 RFID module + white card + blue key tag
- ☐ 1 red LED, 1 green LED
- ☐ 2 resistors, 220 Ω (red-red-brown)
- ☐ Passive buzzer
- ☐ 7 female-to-male Dupont wires, jumper wires
- ☐ Extension: SG90 servo

1. What is RFID?

RFID stands for **R**adio **F**requency **ID**entification. It's inside bus passes, library books, hotel key cards and pet microchips.

The reader has a coil of wire (an **antenna**) that sends out a radio wave. The card also has a coil and a tiny chip, but **no battery**. When the card comes close, the radio wave makes enough electricity in its coil to wake the chip up. The chip then whispers back its number.

> **Big idea: every card has a UID**
>
> Each card has a **UID** (Unique IDentifier), a number set at the factory. Your card's UID is different from your neighbour's. Our program will decide who is allowed in by checking that number.

The UID is written in **hexadecimal** ("hex"), which counts using 16 symbols: `0 1 2 3 4 5 6 7 8 9 A B C D E F`. So a UID looks like `A3 4F 12 9B`.

2. Wire up the reader

3.3 V only!

The RC522 is a **3.3 V** part. Connect its power pin to **3V3**, never to 5V or VIN. 5 V can destroy it.

The RC522 talks to the ESP32 using **SPI**, a fast way of sending data over 4 wires. The ESP32 has special SPI pins, so use exactly these. Use female-to-male Dupont wires from the module's pins to the breadboard.

✓	RC522 pin	ESP32 pin
☐	**3.3V**	**3V3**
☐	**GND**	**GND**
☐	**RST**	**4**
☐	**SDA** (also called SS)	**5**
☐	**SCK**	**18**
☐	**MISO**	**19**
☐	**MOSI**	**23**
☐	**IRQ**	*not connected*

Install the library

Tools → Manage Libraries…, search **MFRC522**, and install **MFRC522 by GithubCommunity**.

3. Sketch 1: Read your card

Type this in and save it as **L17_ReadCard**. Upload it, open the **Serial Monitor** (115200 baud) and hold the card flat on the reader.

Type this in → save it as **L17_ReadCard** — 37 lines

```
// Lesson 17: Read an RFID card
// RC522 reader on SPI: SCK 18, MISO 19, MOSI 23, SDA 5, RST 4.

#include <SPI.h>
#include <MFRC522.h>

const int SS_PIN = 5;
const int RST_PIN = 4;

MFRC522 rfid(SS_PIN, RST_PIN);

String readUid() {
  String uid = "";
  for (int i = 0; i < rfid.uid.size; i++) {
    if (i > 0) uid += " ";
```

```
    if (rfid.uid.uidByte[i] < 16) uid += "0";
    uid += String(rfid.uid.uidByte[i], HEX);
  }
  uid.toUpperCase();
  return uid;
}

void setup() {
  Serial.begin(115200);
  SPI.begin();
  rfid.PCD_Init();
  Serial.println("Hold a card or tag near the reader...");
}

void loop() {
  if (!rfid.PICC_IsNewCardPresent()) return;
  if (!rfid.PICC_ReadCardSerial()) return;

  Serial.print("Card UID: ");
  Serial.println(readUid());
  rfid.PICC_HaltA();
}
```

Write down the UIDs of your card and your key tag. You'll need them next!

Card	UID
White card	
Blue key tag	

How it works

Code	What it means
`MFRC522 rfid(SS_PIN, RST_PIN);`	Make a reader object called `rfid`
`SPI.begin();` `rfid.PCD_Init();`	Start SPI, then start the reader
`if (!rfid.PICC_IsNewCardPresent()) return;`	No card? Then `return`: stop this loop and start again. `!` means **not**
`rfid.uid.uidByte[i]`	Byte number `i` of the UID (an array of numbers)
`String(…, HEX)`	Turn a number into hex text, like `9b`
`uid += " ";`	Stick more text onto the end of the String
`uid.toUpperCase();`	`9b` becomes `9B`
`rfid.PICC_HaltA();`	Tell the card "thanks, go to sleep" so it's only read once

Try it

1. How far away can you hold the card and still read it?
2. Does it work through your hand? Through a book? Through a metal tin lid?
3. Try a bank card or a bus pass (only reading, it's safe). Do they have UIDs too?

4. Add lights and sound

Keep the reader connected and add:

✓	From	To
☐	ESP32 pin **25**	220 Ω resistor → **long leg** of the **red** LED
☐	ESP32 pin **27**	220 Ω resistor → **long leg** of the **green** LED
☐	both LED **short legs**	**–** rail
☐	ESP32 pin **33**	passive buzzer **+**
☐	buzzer **–**	**–** rail

5. Sketch 2: Access control

Save your first sketch as **L17_Access** and change it to this. **Replace the two UIDs** on the `allowed` line with the ones you wrote down. Keep the quotes and the spaces exactly.

Type this in → save it as **L17_Access** 64 lines

```
// Lesson 17: Access control
// Allowed cards light green and beep once, others red and buzz.

#include <SPI.h>
#include <MFRC522.h>

const int SS_PIN = 5;
const int RST_PIN = 4;
const int RED_PIN = 25;
const int GREEN_PIN = 27;
const int BUZZER_PIN = 33;

// Put your own card UIDs here (copy them from L17_ReadCard)
String allowed[] = {"A3 4F 12 9B", "04 7E 2C 1A"};
const int NUM_ALLOWED = 2;

MFRC522 rfid(SS_PIN, RST_PIN);

String readUid() {
  String uid = "";
  for (int i = 0; i < rfid.uid.size; i++) {
    if (i > 0) uid += " ";
    if (rfid.uid.uidByte[i] < 16) uid += "0";
```

```
    uid += String(rfid.uid.uidByte[i], HEX);
  }
  uid.toUpperCase();
  return uid;
}

bool isAllowed(String uid) {
  for (int i = 0; i < NUM_ALLOWED; i++) {
    if (uid == allowed[i]) return true;
  }
  return false;
}

void setup() {
  Serial.begin(115200);
  pinMode(RED_PIN, OUTPUT);
  pinMode(GREEN_PIN, OUTPUT);
  SPI.begin();
  rfid.PCD_Init();
}

void loop() {
  if (!rfid.PICC_IsNewCardPresent()) return;
  if (!rfid.PICC_ReadCardSerial()) return;

  String uid = readUid();
  Serial.print(uid);
  if (isAllowed(uid)) {
    Serial.println("  -> Welcome!");
    digitalWrite(GREEN_PIN, HIGH);
    tone(BUZZER_PIN, 1500, 150);
  } else {
    Serial.println("  -> Access denied");
    digitalWrite(RED_PIN, HIGH);
    tone(BUZZER_PIN, 200, 800);
  }
  delay(2000);
  digitalWrite(GREEN_PIN, LOW);
  digitalWrite(RED_PIN, LOW);
  rfid.PICC_HaltA();
}
```

How it works

Code	What it means
`String allowed[] = {"…", "…"};`	An **array of Strings**: the list of cards that may enter
`bool isAllowed(String uid)`	A function that answers **true** or **false**
`if (uid == allowed[i]) return true;`	`==` compares two Strings. Found it? Answer true straight away
`return false;`	Checked the whole list and no match: answer false
`tone(BUZZER_PIN, 1500, 150);`	High beep for 150 ms. A low 200 Hz buzz sounds like "nope!"

Every card says denied?

Print the UID and compare it letter by letter with your list. Look for `0` vs `O`, missing spaces, or lowercase letters.

6. Challenges

Challenge 1: Add a friend

Add a third card (borrow a friend's) to the list. Remember to change `NUM_ALLOWED` too!

Challenge 2: Say hello

Make a second array `names[]` with a name for each card, and print `Welcome, Sam!` with the right name.

Challenge 3: Servo door

Add a servo as a door lock: it turns to 90° for 3 seconds when an allowed card is shown. Pin 18 is busy (SPI clock), so put the servo's signal wire on pin **13**, red on **5V**, brown on **GND**.

Bonus: Intruder alarm

Count wrong cards in a row. After 3 wrong cards, flash the red LED and sound a siren.

7. Quiz

Check what you learned

1. Why doesn't an RFID card need a battery?

2. Why must the RC522 be connected to 3V3 and not 5V?

3. What does `return` do inside `loop()` ?

4. Is a UID a good password? Why or why not? *(Hint: could someone copy it?)*

Tidy up: unplug the USB cable. Put the card and tag back in the kit. You can leave the reader wired up.

LESSON 18

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Remote Control

Use invisible infrared light to control your ESP32 from across the room.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How an **infrared (IR)** remote sends messages with flashes of light
- How to **decode** remote buttons into code numbers
- How to choose between many options with **switch / case**
- How to make your own remote with the IR transmitter (challenge)

SESSION PLAN

0:00	Invisible light: look through a phone camera
0:10	Wire the IR receiver
0:20	Sketch 1: **L18_ReadCodes** — map every button
0:50	Short break
0:55	Add the RGB LED
1:05	Sketch 2: **L18_RemoteLights**
1:30	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, 2 breadboards
- ☐ IR receiver module
- ☐ Remote control (with its battery)
- ☐ RGB LED
- ☐ 3 resistors, 220 Ω (red-red-brown)
- ☐ 3 female-to-male Dupont wires, jumper wires
- ☐ Challenge: IR transmitter module, 2 buttons

1. Invisible light

A TV remote has a small LED on the front, but when you press a button you can't see it light up. That's because it gives off **infrared (IR)** light, a colour just past red that human eyes can't see.

> **Try it: see the invisible**
>
> Point the remote at a phone camera (the front selfie camera often works best) and press a button. You'll see the LED flicker on the screen! Cameras can see a little infrared.

The remote doesn't just switch the light on. It **flashes it in a pattern** about 38,000 times a second, like super-fast Morse code. Each button has its own pattern. The receiver turns the pattern back into a number.

> **Big idea: a protocol is an agreed language**
>
> The remote and receiver both follow the same rules, called a **protocol**. Our remote uses the **NEC** protocol: every button press sends an **address** (which device) and a **command** (which button).

2. Wire the IR receiver

The IR receiver module has 3 pins. Look closely at the labels. They are usually **–** (or G), **+** (or V, middle) and **S** (signal).

✓	IR receiver pin	ESP32 pin
☐	**–** / G	**GND**
☐	**+** / V (middle)	**3V3**
☐	**S** / signal	**14**

> **Install the library**
>
> **Tools** → **Manage Libraries…**, search **IRremote**, and install **IRremote by shirriff, z3t0, ArminJo**.

3. Sketch 1: What does each button say?

Type this in and save it as **L18_ReadCodes**. Upload it and open the **Serial Monitor** (115200 baud). Pull the plastic tab out of the remote's battery, point it at the receiver and press some buttons.

Type this in → save it as **L18_ReadCodes** — 21 lines

```
// Lesson 18: Read remote control codes
// IR receiver signal pin on 14. Press buttons and read the codes.

#include <IRremote.hpp>

const int IR_PIN = 14;

void setup() {
  Serial.begin(115200);
  IrReceiver.begin(IR_PIN);
  Serial.println("Press a button on the remote...");
}

void loop() {
  if (IrReceiver.decode()) {
    int code = IrReceiver.decodedIRData.command;
    Serial.print("Button code: 0x");
    Serial.println(code, HEX);
    IrReceiver.resume();
  }
}
```

Fill in the code for each button. **Hold** a button down. What do you notice?

Button	Code	Button	Code	Button	Code
POWER	0x_____	1	0x_____	6	0x_____
VOL+	0x_____	2	0x_____	7	0x_____
VOL–	0x_____	3	0x_____	8	0x_____
▶‖	0x_____	4	0x_____	9	0x_____
UP	0x_____	5	0x_____	0	0x_____
DOWN	0x_____	EQ	0x_____	ST/REPT	0x_____

How it works

Code	What it means
`#include <IRremote.hpp>`	Use the IRremote library (note: `.hpp`, not `.h`)
`IrReceiver.begin(IR_PIN);`	Start listening for IR on pin 14
`if (IrReceiver.decode())`	**True** when a full message has arrived and been decoded
`IrReceiver.decodedIRData.command`	The button's command number
`Serial.println(code, HEX);`	Print the number in **hexadecimal**, like `45` instead of `69`
`0x45`	How we write a hex number in code. `0x45` = 69
`IrReceiver.resume();`	"I'm done with that one, get ready for the next"

Getting 0x0 or weird numbers?

Make sure the remote's battery tab is out. Bright sunlight or lamps can confuse the receiver. Point the remote straight at it, from less than 2 metres.

4. Add the RGB LED

✓	From	To
☐	ESP32 pin **25**	220 Ω resistor → RGB **red** leg
☐	ESP32 pin **26**	220 Ω resistor → RGB **green** leg
☐	ESP32 pin **27**	220 Ω resistor → RGB **blue** leg
☐	RGB **longest leg** (common –)	**–** rail

5. Sketch 2: Remote control lights

Save as **L18_RemoteLights** and type this. If your codes in the table are different from the ones below, **use your codes**.

Type this in → save it as **L18_RemoteLights** 38 lines

```
// Lesson 18: Remote control lights
// Buttons 1–5 pick a colour on the RGB LED, POWER turns it off.

#include <IRremote.hpp>

const int IR_PIN = 14;
const int RED_PIN = 25;
const int GREEN_PIN = 26;
const int BLUE_PIN = 27;

void color(int r, int g, int b) {
  analogWrite(RED_PIN, r);
  analogWrite(GREEN_PIN, g);
  analogWrite(BLUE_PIN, b);
}

void setup() {
  Serial.begin(115200);
  IrReceiver.begin(IR_PIN);
  color(0, 0, 0);
}

void loop() {
  if (IrReceiver.decode()) {
    int code = IrReceiver.decodedIRData.command;
    Serial.println(code, HEX);

    switch (code) {
      case 0x0C: color(255, 0, 0);     break;  // 1 = red
      case 0x18: color(0, 255, 0);     break;  // 2 = green
      case 0x5E: color(0, 0, 255);     break;  // 3 = blue
      case 0x08: color(255, 120, 0);   break;  // 4 = yellow
      case 0x1C: color(255, 255, 255); break;  // 5 = white
      case 0x45: color(0, 0, 0);       break;  // POWER = off
    }
    IrReceiver.resume();
  }
}
```

How it works

Code	What it means
`switch (code) { … }`	Look at `code` and jump to the matching `case`
`case 0x0C: … break;`	If `code` is `0x0C`, do this, then `break` out of the switch
(no matching case)	Nothing happens. Other buttons are ignored
`color(255, 120, 0);`	Our function from the PWM lesson: red, green, blue from 0 to 255

switch vs. lots of ifs

`switch` does the same job as `if (code == 0x0C) … else if (code == 0x18) …`, but is easier to read when there are many choices. Don't forget the `break`! Without it, the code "falls through" into the next case.

Try it

1. Add buttons 6, 7, 8 and 9 with your own colours (purple, cyan, pink…).
2. Does it work if you bounce the IR off a wall or the ceiling?
3. Try your TV remote at home (or a friend's). Do its buttons give different codes?

6. Challenges

Challenge 1: Brighter, dimmer

Make **VOL+** and **VOL–** change the brightness. *Hint: keep a* `brightness` *variable from 0 to 100, and use* `r * brightness / 100` *in* `analogWrite`.

Challenge 2: Power toggle

Make **POWER** switch the light off **and** back on to the same colour. *Hint: a* `bool isOn` *and* `isOn = !isOn;`

Challenge 3: Party mode

Button **0** starts random colours every 150 ms until another button is pressed. Use `random(256)`.

Bonus: Build your own remote

The kit has an **IR transmitter** module too. With a friend, wire it to one ESP32 (signal on pin **4**, power to 3V3, GND to GND) plus two buttons on 13 and 14. Use `IrSender.begin(4);` and `IrSender.sendNEC(0x00, 0x0C, 0);` to send "button 1" to your friend's ESP32!

7. Quiz

Check what you learned

1. Why can't you see the light from a remote control?

2. What does `break` do in a `switch`?

3. What number is `0x10` in normal (decimal) numbers? *(Hint: hex counts in sixteens)*

4. Name two things in your house that might use IR remotes.

Tidy up: unplug the USB cable and **put the plastic tab back in the remote** (or take out the battery) so it doesn't run flat.

LESSON 19

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Tilt Maze

Feel gravity with a motion sensor and build a tilt-to-play game.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- What an **accelerometer** is and how it feels gravity
- How to talk to a chip over **I²C** with the `Wire` library
- How to make a **negative number** out of two bytes
- How to build a game: moving sprites, **collisions** and a score

SESSION PLAN

0:00	How does a phone know which way is up?
0:10	Wire the GY-6500 and the OLED
0:25	Sketch 1: **L19_Tilt** + Serial Plotter
0:50	Short break
0:55	Sketch 2: **L19_RollingBall**
1:15	Sketch 3: **L19_DotGame** (5 changes)
1:35	Challenges: build a maze
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, 2 breadboards
- ☐ GY-6500 motion sensor module
- ☐ 0.96" OLED display
- ☐ Passive buzzer
- ☐ Jumper wires

1. Feeling gravity

When you turn your phone sideways, the screen rotates. How does the phone know? It has an **accelerometer** inside: a tiny chip with a microscopic weight on springs. When you tilt it, gravity pulls the weight to one side, and the chip measures how far.

Your kit has one too: the **GY-6500** module. It measures in 3 directions, called **axes**: **x** (left ↔ right), **y** (forward ↔ back) and **z** (up ↕ down).

> **Big idea: 1 g = 16384**
>
> Lying flat, gravity pulls straight down, so **z** reads about **16384**. That number means "1 g", the pull of Earth's gravity. Tilt the sensor and some of that pull moves to x or y. The numbers can be **negative** too: tilting left gives a negative x, tilting right a positive x.

2. Wire it up

The GY-6500 and the OLED both use **I²C**, a way for many chips to share the same two wires, **SDA** (data) and **SCL** (clock). Each chip has its own **address**: the OLED is `0x3C`, the GY-6500 is `0x68`. So you connect both to the same pins!

Sharing wires

Put the two SDA wires in the same breadboard row as ESP32 pin 21, and the two SCL wires in the same row as pin 22.

✓	From	To
☐	GY-6500 **VCC**	**3V3**
☐	GY-6500 **GND**	**GND**
☐	GY-6500 **SDA**	ESP32 pin **21**
☐	GY-6500 **SCL**	ESP32 pin **22**
☐	OLED **VCC**, **GND**	**3V3**, **GND**
☐	OLED **SDA**, **SCL**	pins **21**, **22** (same rows as above)
☐	Buzzer **+** / **–**	pin **33** / **GND**

Leave the other GY-6500 pins (EDA, ECL, AD0, INT, NCS, FSYNC) empty. Push the GY-6500 into the breadboard **flat and level**: it's going to be your game controller!

3. Sketch 1: Read the tilt

Type this in and save as **L19_Tilt**. Upload, then open **Tools** → **Serial Plotter** (115200 baud) and tilt your breadboard.

What's a register?

Think of the chip as a row of numbered lockers, called **registers**. Some lockers hold settings (like locker 0x6B, the on/off switch). Others hold measurements (lockers 0x3B to 0x40 hold the tilt). We read and write lockers by their number.

Type this in → save it as **L19_Tilt** 45 lines

```
// Lesson 19: Read the tilt sensor
// GY-6500 on I2C (SDA 21, SCL 22). Open the Serial Plotter!

#include <Wire.h>

const int IMU = 0x68;
int ax, ay, az;

int readAxis() {
  int high = Wire.read();
  int low = Wire.read();
  int value = high * 256 + low;
  if (value > 32767) value = value - 65536;
  return value;
}

void readAccel() {
```

```
  Wire.beginTransmission(IMU);
  Wire.write(0x3B);               // first accelerometer register
  Wire.endTransmission(false);
  Wire.requestFrom(IMU, 6);
  ax = readAxis();
  ay = readAxis();
  az = readAxis();
}

void setup() {
  Serial.begin(115200);
  Wire.begin();
  Wire.beginTransmission(IMU);
  Wire.write(0x6B);               // power register
  Wire.write(0);                  // 0 = wake up!
  Wire.endTransmission();
}

void loop() {
  readAccel();
  Serial.print("x:");
  Serial.print(ax);
  Serial.print(" y:");
  Serial.print(ay);
  Serial.print(" z:");
  Serial.println(az);
  delay(100);
}
```

How it works

Code	What it means
`Wire.begin();`	Start I²C on pins 21 (SDA) and 22 (SCL)
`Wire.beginTransmission(IMU);` … `Wire.endTransmission();`	Send a message to the chip at address `0x68`
`Wire.write(0x6B); Wire.write(0);`	"Set **register** 0x6B to 0." The chip starts asleep; this wakes it up
`Wire.write(0x3B);`	"I want to read, starting at register 0x3B" (where x is stored)
`Wire.requestFrom(IMU, 6);`	Ask for 6 bytes: x, y and z, two bytes each
`high * 256 + low`	Glue two bytes (0–255) into one number (0–65535)
`if (value > 32767) value = value - 65536;`	Numbers over half-way are really **negative**. This is called **two's complement**

Try it

1. Lay it flat. What does **z** say? Turn the breadboard upside down. What now?
2. Which way do you tilt to make **x** go positive? And **y**?
3. Shake it! What happens to the lines?

4. Sketch 2: Rolling ball

Save as **L19_RollingBall** and type this in. Upload it and tilt the breadboard to roll the ball.

Type this in → save it as **L19_RollingBall** — 53 lines

```
// Lesson 19: Rolling ball
// Tilt the breadboard to roll a ball around the OLED screen.

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

const int IMU = 0x68;
int ax, ay;
float ballX = 64;
float ballY = 32;

int readAxis() {
  int high = Wire.read();
  int low = Wire.read();
  int value = high * 256 + low;
  if (value > 32767) value = value - 65536;
  return value;
}

void readAccel() {
  Wire.beginTransmission(IMU);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(IMU, 4);
  ax = readAxis();
  ay = readAxis();
}

void setup() {
  Wire.begin();
  Wire.beginTransmission(IMU);
  Wire.write(0x6B);
  Wire.write(0);
  Wire.endTransmission();
```

```
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
  readAccel();
  ballX = ballX - ax / 2000.0;   // wrong way? change - to +
  ballY = ballY + ay / 2000.0;
  ballX = constrain(ballX, 3, 124);
  ballY = constrain(ballY, 3, 60);

  display.clearDisplay();
  display.drawRect(0, 0, 128, 64, WHITE);
  display.fillCircle(ballX, ballY, 3, WHITE);
  display.display();
  delay(20);
}
```

We only need x and y now, so we ask for **4** bytes instead of 6.

Code	What it means
`float ballX = 64;`	A **float** can hold decimals like 64.37, so the ball moves smoothly
`ballX = ballX - ax / 2000.0;`	Tilt more and the ball moves further each frame. `2000.0` controls the speed
`constrain(ballX, 3, 124)`	Keep the ball inside the screen
`fillCircle(x, y, 3, WHITE)`	Draw a filled circle with radius 3

Ball rolls the wrong way? It depends how your sensor sits. Change the `-` to a `+` (or the other way) on the `ballX` or `ballY` line.

5. Sketch 3: Collect the dots

Now let's turn it into a game. You don't need to type it all again! Use **File → Save As…** to save **L19_RollingBall** as **L19_DotGame**, then make these **5 changes**. New lines are shown in the boxes.

> **Big idea: collision detection**
>
> Every video game checks "are these two things touching?" many times a second. It's called **collision detection**. Here we check if the gaps in x and y are both small.

Change 1: under `float ballY = 32;` add:

```
const int BUZZER_PIN = 33;
int dotX, dotY;
int score = 0;
```

Change 2: above `void setup()` add a new function:

```
void newDot() {
  dotX = random(8, 120);
  dotY = random(12, 56);
}
```

Change 3: at the end of `setup()`, under `display.begin(…);` add:

```
  display.setTextColor(WHITE);
  newDot();
```

Change 4: in `loop()`, under `ballY = constrain(ballY, 3, 60);` add:

```
  if (abs(ballX - dotX) < 5 && abs(ballY - dotY) < 5) {
    score = score + 1;
    tone(BUZZER_PIN, 1500, 60);
    newDot();
  }
```

Change 5: under `display.drawRect(0, 0, 128, 64, WHITE);` add:

```
  display.setCursor(4, 4);
  display.print(score);
  display.drawCircle(dotX, dotY, 2, WHITE);
```

Upload and play! Stuck? Check each change is in the right place, inside the right `{ }`.

Code	What it means
`random(8, 120)`	A random whole number from 8 up to 119
`abs(ballX - dotX) < 5`	`abs` removes the minus sign. True if the ball and dot are less than 5 pixels apart
`&&`	**and**: both must be true. The ball must be close in x **and** y

6. Challenges

Challenge 1: Speed run

Add a 30-second timer using `millis()`. When time is up, show `GAME OVER` and the score, wait 3 seconds, and restart.

Challenge 2: Danger dot

Add a second dot (draw it as a square) that **loses** a point if you touch it. Make it move to a new place every 3 seconds.

Challenge 3: Tilt maze

Draw walls with `display.fillRect(x, y, w, h, WHITE)`. Stop the ball moving into a wall, and give the player a goal square to reach. Show how many seconds it took!

Bonus: Spirit level

Make a level for hanging pictures: show "LEVEL" on the screen and beep when `ax` is between -300 and 300.

7. Quiz

Check what you learned

1. Lying flat on the table, which axis feels gravity?

2. Why can the OLED and GY-6500 share the same two wires?

3. What does `&&` mean in an `if`?

4. Name a game or device that uses an accelerometer.

Tidy up: unplug the USB cable. Leave the OLED wired; you'll use it again in your final project.

LESSON 20

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Wi-Fi Control Panel

Turn your ESP32 into a tiny website you can control from your phone.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How the ESP32 can make its **own Wi-Fi network**
- What an **IP address** is and how a browser finds a web page
- How to write a simple web page in **HTML**
- How a **web server** answers different addresses like `/on` and `/off`

SESSION PLAN

0:00	How does the internet find a web page?
0:10	Wire the LED and knob
0:20	Sketch 1: **L20_HelloWeb** — your first web page
0:50	Short break
0:55	Sketch 2: **L20_LedControl** — ON/OFF buttons
1:25	Challenges: design your control panel
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, 2 breadboards
- ☐ 1 LED + 220 Ω resistor
- ☐ Potentiometer (10 kΩ knob)
- ☐ Jumper wires
- ☐ A phone, tablet or laptop with Wi-Fi (one per group)

1. How does a web page reach you?

When you type a website into a browser, your device sends a message: "*please send me your page*". A computer called a **web server** answers by sending back the page, written in a language called **HTML**. Your browser draws it on screen.

Your ESP32 is powerful enough to be a web server! And it can even make its **own Wi-Fi network**, called an **access point**, so you don't need the school's Wi-Fi.

> **Big idea: IP addresses are like house numbers**
>
> Every device on a network has an **IP address**, like `192.168.4.1`. It's like a house number: it tells messages where to go. When your ESP32 makes its own network, it always gives itself the address **192.168.4.1**. Type that into a browser and you're knocking on your ESP32's door.

2. Wire it up

✓	From	To
☐	ESP32 pin **26**	220 Ω resistor → **long leg** of the LED
☐	LED **short leg**	**–** rail
☐	Knob **left** leg	**GND**
☐	Knob **middle** leg	ESP32 pin **34**
☐	Knob **right** leg	**3V3**

3. Sketch 1: Hello, web!

Type this in and save as **L20_HelloWeb**.

> **Pick a unique Wi-Fi name!**
>
> Everyone in the room is making a Wi-Fi network. Change `Sparky-ESP32` to your own name, like `Maya-ESP32`, so you can find yours. The password must be **at least 8 characters**.

> **A tiny bit of HTML**
>
> HTML uses **tags** in angle brackets. Most come in pairs: `<h1>` starts a big heading and `</h1>` ends it. `<p>…</p>` is a paragraph. Our sketch builds the page as a **String**, adding one piece at a time with `+=`.

Type this in → save it as **L20_HelloWeb** — 36 lines

```
// Lesson 20: Hello, web!
// The ESP32 makes its own Wi-Fi network and serves a web page.

#include <WiFi.h>
#include <WebServer.h>

String WIFI_NAME = "Sparky-ESP32";   // make this unique!
String PASSWORD = "secret123";       // at least 8 characters
const int POT_PIN = 34;

WebServer server(80);

void handleRoot() {
  String page = "<html><body style='font-family:sans-serif'>";
  page += "<h1>Hello from my ESP32!</h1>";
  page += "<p>Knob reading: ";
  page += analogRead(POT_PIN);
  page += "</p></body></html>";
  server.send(200, "text/html", page);
}

void setup() {
```

```
  Serial.begin(115200);
  WiFi.softAP(WIFI_NAME, PASSWORD);
  Serial.print("Join Wi-Fi ");
  Serial.print(WIFI_NAME);
  Serial.print(" then open http://");
  Serial.println(WiFi.softAPIP());

  server.on("/", handleRoot);
  server.begin();
}

void loop() {
  server.handleClient();
}
```

Now connect:

- ☐ Upload, and open the **Serial Monitor** (115200 baud). Press **EN** if nothing shows.
- ☐ On your phone, open **Wi-Fi settings** and join your network. Type your password.
- ☐ Your phone may say "*No internet*". That's OK, stay connected!
- ☐ Open a browser and type **192.168.4.1** in the address bar.
- ☐ Turn the knob and **refresh** the page. The number changes!

How it works

Code	What it means
`WiFi.softAP(WIFI_NAME, PASSWORD);`	Make a Wi-Fi network (a **soft Access Point**)
`WiFi.softAPIP()`	The ESP32's own IP address: 192.168.4.1
`WebServer server(80);`	A web server. Web pages use "door number" (**port**) 80
`server.on("/", handleRoot);`	When someone asks for the main page `/`, run `handleRoot()`
`server.send(200, "text/html", page);`	Send the page back. **200** means "OK, here it is"
`server.handleClient();`	Check for visitors. Must run all the time, so it lives in `loop()`

Try it

1. Change the heading to say something about you.
2. Add another line: `page += "<p>I am 12 years old</p>";` (or anything you like).
3. Ask a friend to join **your** network and visit your page. Does it work on two phones at once?

4. Sketch 2: Buttons that control the LED

Save as **L20_LedControl** and type this in. The Wi-Fi name and password stay the same as yours.

Type this in → save it as **L20_LedControl** 58 lines

```
// Lesson 20: Control an LED from your phone
// Buttons on the web page switch the LED on pin 26.

#include <WiFi.h>
#include <WebServer.h>

String WIFI_NAME = "Sparky-ESP32";
String PASSWORD = "secret123";
const int POT_PIN = 34;
const int LED_PIN = 26;

WebServer server(80);
bool ledOn = false;

void sendPage() {
  String page = "<html><head><meta name='viewport' ";
  page += "content='width=device-width'></head>";
  page += "<body style='font-family:sans-serif;text-align:center'>";
  page += "<h1>My ESP32</h1>";
  page += "<p>LED is ";
  if (ledOn) page += "ON";
  else page += "OFF";
  page += "</p><p>Knob: ";
  page += analogRead(POT_PIN);
  page += "</p>";
  page += "<a href='/on'><button>ON</button></a> ";
  page += "<a href='/off'><button>OFF</button></a>";
  page += "</body></html>";
  server.send(200, "text/html", page);
}

void handleOn() {
  ledOn = true;
  digitalWrite(LED_PIN, HIGH);
  sendPage();
}

void handleOff() {
  ledOn = false;
  digitalWrite(LED_PIN, LOW);
  sendPage();
}

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  WiFi.softAP(WIFI_NAME, PASSWORD);
```

```
  Serial.println(WiFi.softAPIP());

  server.on("/", sendPage);
  server.on("/on", handleOn);
  server.on("/off", handleOff);
  server.begin();
}

void loop() {
  server.handleClient();
}
```

Join your network again and go to **192.168.4.1**. Press **ON** and **OFF**. Watch the LED!

How it works

Code	What it means
`<a href='/on'><button>ON</button></a>`	A button that is also a **link**: pressing it visits the address `/on`
`server.on("/on", handleOn);`	When a browser visits `/on`, run `handleOn()`
`digitalWrite(LED_PIN, HIGH);` then `sendPage();`	Switch the LED on, then send the page back so the phone shows the new state
`<meta name='viewport' …>`	Tells phones to show the page at a sensible size
`'` inside `"…"`	HTML needs quotes too. We use single `'` quotes inside the String's double `"` quotes

Look at the address bar

After pressing ON, the address says `192.168.4.1/on`. Every web page you visit works this way: the part after the `/` tells the server *what* you want.

Page won't load?

Check your phone is still on the ESP32's network; some phones switch back to a network that has internet. Turning off **mobile data** can help.

5. Challenges

Challenge 1: Make it pretty

Add colour with **styles**. Try `<button style='font-size:30px;background:green;color:white'>`. Change the page background with `<body style='background:lightyellow'>`.

Challenge 2: Beep button

Add a buzzer on pin **33** and a **BEEP** button that visits `/beep` and plays a tone.

Challenge 3: Auto-refresh

Make the knob reading update by itself. Add this inside `<head>`: `<meta http-equiv='refresh' content='2; url=/'>`. What does the `2` do?

Bonus: Brightness levels

Make **LOW**, **MEDIUM** and **HIGH** buttons that use `analogWrite` to set the LED brightness. *Hint: you can visit `/dim?level=40` and read the number with `server.arg("level").toInt()`.*

6. Quiz

Check what you learned

1. What IP address does the ESP32 give itself in access-point mode?

   _____

2. What does `server.on("/off", handleOff);` mean, in your own words?

   _____

   _____

3. What is the difference between `<h1>` and `</h1>`?

   _____

4. Why should everyone in the class use a different Wi-Fi name?

   _____

Tidy up: unplug the USB cable, and on your phone choose **Forget this network** so it reconnects to your normal Wi-Fi.

LESSON 21

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

My Invention

Plan, build, test and show off a project of your own design.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How real engineers go from an **idea** to a working **prototype**
- How to combine **inputs** and **outputs** from earlier lessons
- How to build and test **one piece at a time**
- How to explain your invention to others

SESSION PLAN

0:00	The inventor's process + idea menu
0:15	Example: the Smart Room Monitor
0:30	Fill in your planning sheet
0:50	Short break
0:55	Build and test, one piece at a time
1:40	Show and tell
1:55	Reflection and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, 2 breadboards
- ☐ Whatever your project needs (see your plan!)
- ☐ For the example: DHT11, PIR sensor, OLED, passive buzzer
- ☐ Your old sketches from lessons 1–20
- ☐ A pencil

1. You are the inventor now

You've learned about LEDs, buttons, sounds, screens, sensors, motors, cards, remotes and Wi-Fi. Today **you** decide what to build!

> **Big idea: the inventor's loop**
>
> Real engineers don't build everything at once. They follow a loop: **Idea** → **Plan** → **Build a small piece** → **Test** → **Fix** → **Build the next piece…** and finally **Show**. If something goes wrong, that's normal. It's just the *Test* → *Fix* part of the loop.

Every invention is made of **inputs** (things that sense the world) and **outputs** (things that do something):

Inputs (sense)	Outputs (do)
Button, keypad, knob, joystick, tilt switch	LED, RGB LED, 7-segment display
Light sensor, temperature (DHT11), thermistor	Buzzer: beeps and tunes
Distance (ultrasonic), motion (PIR), tilt (GY-6500)	OLED screen: words, numbers, pictures
RFID card, IR remote, Wi-Fi web page	Servo, stepper motor, fan motor

2. Idea menu

Pick one, change one, or invent your own!

Idea	What it does	Main parts	Lessons to look back at
Smart Room Monitor	Shows temperature, warns when too hot, chimes when someone comes in	DHT11, PIR, OLED, buzzer	10, 11, 13
Secret Treasure Box	Opens a servo lock with the right card *or* keypad code	RFID or keypad, servo, LEDs	14, 16, 17
Parking Helper	Beeps faster and shows distance as a car gets closer	Ultrasonic, buzzer, OLED or LEDs	6, 10, 12
Reaction Game	Two players race to press their button after a light	Buttons, LEDs, 4-digit display	3, 9
Plant Guardian	Warns when it's too dark or too hot for a plant	Photoresistor, DHT11, RGB LED	4, 5, 11
Desk Fan Controller	Fan speeds up when it's warmer; remote to turn it off	DHT11, DC motor, IR remote	11, 15, 18
Phone-Controlled Disco	Pick colours and tunes from a web page	RGB LED, buzzer, Wi-Fi	5, 6, 20
Intruder Alarm	Arm with a keypad code; motion sets off a siren	PIR, keypad, buzzer, LEDs	13, 16

3. Worked example: Smart Room Monitor

Here is one finished project, so you can see how the pieces come together. You can build it as it is, or use it as a starting point and change it.

✓	Part	Pin	Power
☐	DHT11 **S** (signal)	**4**	+ to **3V3**, – to **GND**
☐	PIR **OUT** (middle)	**27**	VCC to **5V**, GND to **GND**
☐	Buzzer **+**	**33**	– to **GND**
☐	OLED **SDA / SCL**	**21 / 22**	VCC to **3V3**, GND to **GND**

Type this in → save it as **L21_SmartRoom** 62 lines

```
// Lesson 21: Example project – Smart Room Monitor
// DHT11 on 4, PIR on 27, buzzer on 33, OLED on 21/22.

#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

const int DHT_PIN = 4;
const int PIR_PIN = 27;
```

```
const int BUZZER_PIN = 33;
const float TOO_HOT = 28.0;

Adafruit_SSD1306 display(128, 64, &Wire, -1);
DHT dht(DHT_PIN, DHT11);

float temp = 0;
float humidity = 0;
unsigned long lastRead = 0;
bool wasMotion = false;

void setup() {
  pinMode(PIR_PIN, INPUT);
  dht.begin();
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.setTextColor(WHITE);
}

void loop() {
  // 1. Read the DHT11 every 2 seconds (it is slow)
  if (millis() - lastRead > 2000) {
    temp = dht.readTemperature();
    humidity = dht.readHumidity();
    lastRead = millis();
  }

  // 2. Ding-dong when someone walks in
  bool motion = digitalRead(PIR_PIN) == HIGH;
  if (motion && !wasMotion) {
    tone(BUZZER_PIN, 1200, 150);
    delay(200);
    tone(BUZZER_PIN, 900, 300);
  }
  wasMotion = motion;

  // 3. Show everything on the screen
  display.clearDisplay();
  display.setTextSize(2);
  display.setCursor(0, 0);
  display.print(temp, 1);
  display.print(" C");
  display.setCursor(0, 20);
  display.print(humidity, 0);
  display.print(" %");
  display.setTextSize(1);
```

```
  display.setCursor(0, 44);
  if (temp > TOO_HOT) display.print("Too hot! Open a window");
  display.setCursor(0, 56);
  if (motion) display.print("Someone is here!");
  else display.print("Room is empty");
  display.display();
  delay(100);
}
```

Where did each piece come from?

Part of the sketch	Learned in
`DHT dht(DHT_PIN, DHT11);` and `readTemperature()`	Lesson 11: Weather Station
`if (millis() - lastRead > 2000)` (do something every 2 s without `delay`)	Lesson 9
`digitalRead(PIR_PIN)` and remembering `wasMotion`	Lesson 13: Motion Alarm
`if (motion && !wasMotion)`: only beep at the **moment** motion starts	Lesson 3 (buttons)
`display.print(temp, 1)`: show 1 decimal place	Lesson 10: OLED

Notice the comments

The `loop()` is split into steps: **1. read**, **2. decide**, **3. show**. Most inventions follow this pattern. It's a great way to plan your own sketch!

4. My planning sheet

My invention is called:

What it does (one or two sentences):

Who would use it, and why?

Inputs and outputs:

Inputs (what it senses)	Outputs (what it does)

Parts list and pins:

✓	Part	ESP32 pin(s)	Power (3V3 or 5V)	Lesson to look back at
☐				
☐				
☐				
☐				
☐				

Pin check before you wire

Use pins from your earlier lessons. **Don't use** pins 0, 2, 12 or 15. Pins 34–39 are **input only**. Two parts can't share one pin, except I²C parts (OLED and GY-6500) on 21/22. 5 V parts like the servo and ultrasonic sensor get power from **5V**, but the ESP32 pins only take 3.3 V!

Sketch plan: what will `loop()` do? Write it in plain words first.

Step	What my sketch does
1. **Read** the inputs	
2. **Decide** what to do	
3. **Show / do** something	

5. Build and test, one piece at a time

The golden rule

Never wire everything and write everything at once. Add **one** part, get it working, then add the next. When something breaks, you'll know it was the last thing you added.

- ☐ Step 1: wire your **first input** and print its value to the Serial Monitor
- ☐ Step 2: wire your **first output** and test it on its own
- ☐ Step 3: connect them with an `if`: *when input does X, output does Y*
- ☐ Step 4: add the next part… and test again
- ☐ Step 5: make it fun: sounds, messages, animations

Test log: write down what you tried, especially when it didn't work!

What I tried	What happened	What I'll change

Stuck?

1. Open the sketch from the lesson that used the part, and check the wiring and code against it.
2. Add `Serial.println(…)` to see what your variables really are.
3. Explain your code out loud to a friend (or a rubber duck!). You'll often spot the problem yourself.

6. Show and tell

Each inventor gets **2 minutes**. Use these prompts:

- [ ] What is it called and what problem does it solve?
- [ ] Demo: show it working!
- [ ] What was the hardest bug, and how did you fix it?
- [ ] What would you add if you had another 2 hours?

Extension: level it up

Finished early? Add a **Wi-Fi web page** (Lesson 20) so your invention can be checked or controlled from a phone. Your teacher has an example that adds this to the Smart Room Monitor.

7. Reflection

Look back

1. Which lesson was your favourite, and why?

   _____

2. What's one thing you can do now that you couldn't do in Lesson 1?

   _____

3. Name one invention you'd love to build next, maybe with parts that aren't in the kit.

   _____

   _____

Tidy up: take a photo of your project! Then carefully put every part back in its place in the kit.
Congratulations, inventor! 🎉